

Thadomal Shahani Engineering College

Bandra (W.), Mumbai - 400 050.

CERTIFICATE

Certify that Mr./Miss Aarya Vijay Arban
of IT Department, Semester VI with
Roll No. 05 has completed a course of the necessary
experiments in the subject BT Lab under my
supervision in the **Thadomal Shahani Engineering College**
Laboratory in the year 2024 - 2025

Teacher In- Charge

Head of the Department

Date 17/04/2025

Principal

~~John~~
12/04/20

Aarya Arban

T11 – 05

Assignment No. 1

Aim: Creating Pivot table using Excel

1. Maximum and Minimum Order Amount for each country, sales person wise

Row Labels	Max of Order Amount
Country	0
Salesperson	0
UK	9210.9
Buchanan	9210.9
Dodsworth	5275.71
King	9194.56
Suyama	4707.54
USA	16387.5
Callahan	4825
Davolio	6635.27
Fuller	16387.5
Leverling	10952.84
Peacock	11188.4
Grand Total	16387.5

Row Labels	Min of Order Amount
Country	0
Salesperson	0
UK	12.5
Buchanan	103.2
Dodsworth	12.5
King	88.8
Suyama	48
USA	18.4
Callahan	48.75
Davolio	72.96
Fuller	40
Leverling	86.4
Peacock	18.4
Grand Total	12.5

2. Total number of orders per country, sales person wise

Row Labels	Count of Order Amount
Country	1
Salesperson	1
UK	217
Buchanan	49
Dodsworth	36
King	65
Suyama	67
USA	581
Callahan	102
Davolio	120
Fuller	84
Leverling	120
Peacock	155
Grand Total	799

3. Average amount for each country

Row Labels	Average of Order Amount
Country	#DIV/0!
Salesperson	#DIV/0!
UK	1526.838824
Buchanan	1729.955636
Dodsworth	1493.05697
King	1624.91334
Suyama	1301.295234
USA	1556.55347
Callahan	1245.138111
Davolio	1571.412625
Fuller	1731.108615
Leverling	1564.84345
Peacock	1648.96533
Grand Total	1548.473172

4. Total order amount for peacock only

Row Labels	Sum of Order Amount
Peacock	255589.6262
Grand Total	255589.6262

5. Who has made the maximum sales in UK

Country		UK	
Row Labels		Max of Order Amount	
King		12615.05	
Grand Total		12615.05	

6. Which salesperson has made the highest sales

Row Labels		Count of Order Amount	
Peacock		151	
Grand Total		151	

7. which country has made more sales

Row Labels		Count of Order Amount	
USA		584	
Grand Total		584	

8. What are total sales for each country

Row Labels		Sum of Order Amount	
UK		333330.91	
USA		894996.49	
Grand Total		1228327.4	

9. Top 2 sales persons who have made the highest sales in all countries

Row Labels	Count of OrderID
UK	132
King	67
Suyama	65
USA	276
Leverling	125
Peacock	151
Grand Total	408

1. Total sales made by each country quarter wise

Sum of Order Amount	Column Labels			
Row Labels	2003	2004	2005	Grand Total
UK	57313.55	156843.61	119173.75	333330.91
USA	136002.99	452347.15	306646.35	894996.49
Grand Total	193316.54	609190.76	425820.1	1228327.4

2. Average sales of sales persons year wise

Average of Order Amount	Column Labels			
Row Labels	2003	2004	2005	Grand Total
Buchanan	1766.72	1654.376842	1514.760769	1637.910714
Callahan	1127.1	1035.527636	1737.701852	1242.754242
Davolio	1341.815652	1711.613571	1468.104474	1559.829829
Dodsworth	1978.902	1375.382778	2244.257778	1830.44
Fuller	1272.247143	1735.808293	1987.14	1766.345435
King	1384.741818	1759.623235	1904.710909	1745.716269
Leverling	1012.442222	1548.045821	1981.331	1609.57016
Peacock	1611.132581	1598.148205	1218.166905	1495.123709
Suyama	1037.12	1316.979677	859.079	1115.809692
Grand Total	1351.863916	1526.793885	1656.887549	1537.330914

3. Monthly sales report of all the sales persons

Sum of Order Amount		Column Labels											
Row Labels	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Grand Total
Buchanan	9816.52	7980.06	4922.43	210	4500.27	2147.4	8362.02	2076.2	5596.4	8974.52	4385.33	9821.1	68792.25
Callahan	20456.89	6853.36	19712.83	18493.6	8157.9	1922.32	1255.34	15116.85	4317.8	13541.5	6021.55	7182.73	123032.67
Davolio	20628.45	7642.62	15965.75	30855.97	9355.96	5150.6	10281.55	19697.89	11307.95	15416.45	15348.8	20848.1	182500.09
Dodsworth	6660.62	20418.34	5401.05	10881.61	555.6	3482.5	4388.1		5261.25	10325.11	7495.36	178.5	75048.04
Fuller	13929.19	17036.5	17351.25	37052.17	14654.62	7071.5	10082.5	2554.3	7655.6	17169.85	8221.3	9725	162503.78
King	18279.34	6509.13	10077.35	29947.25	5871.53	2765.3	3569.58	8484.39	9895.96	9522.98	10977.98	1062.2	116962.99
Leverling	28693.32	35656.39	25854.72	18093.96	14180.45	18674.38	4840.18	5191.83	7272.25	12875.89	9775.23	20087.67	201196.27
Peacock	22243.03	39644.71	19396.03	17383.87	10569.34	5404.17	13119.54	22083.11	11618.39	24473.08	12075.71	27752.7	225763.68
Suyama	7690.67	4850.14	6362.48	11579.95	3061.9	3464.81	2356.4	6308.16	5611.83	5697.5	5602.75	9941.04	72527.63
Grand Total	148398.03	146591.25	125043.89	174498.38	70907.57	50082.98	58255.21	81512.73	68537.43	117996.88	79904.01	106599.04	1228327.4

Aarya Arban

T11 – 05

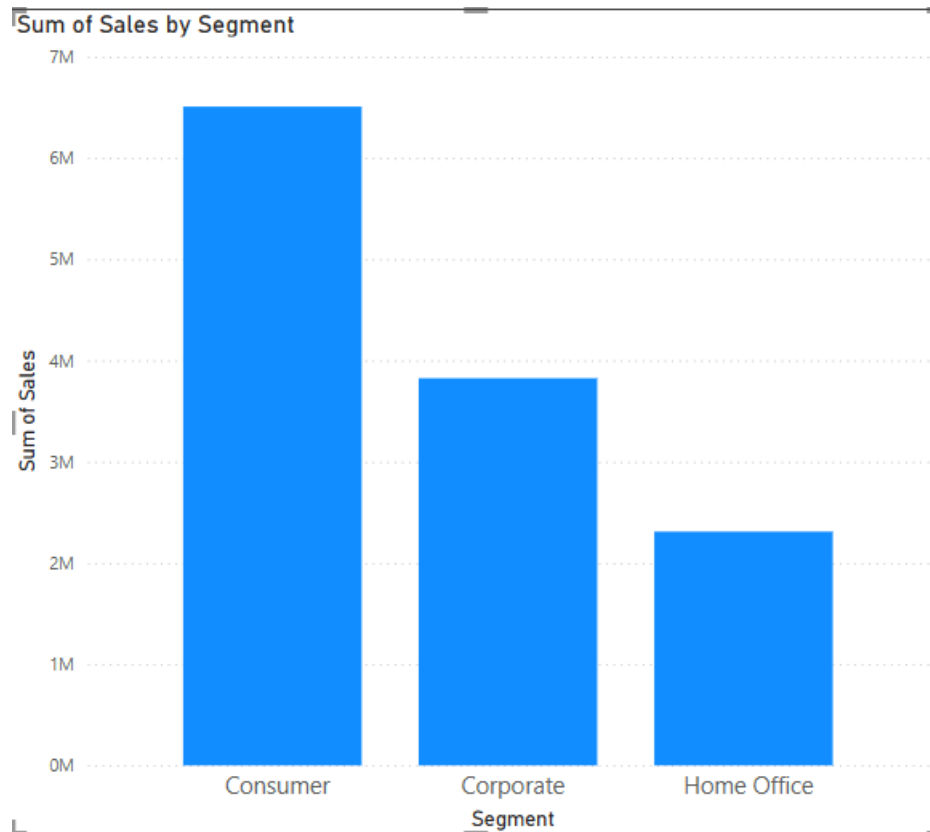
Assignment No. 2

Aim: Building dashboard using PowerBI charts for strategic query analysis on superstore dataset and its visualization.

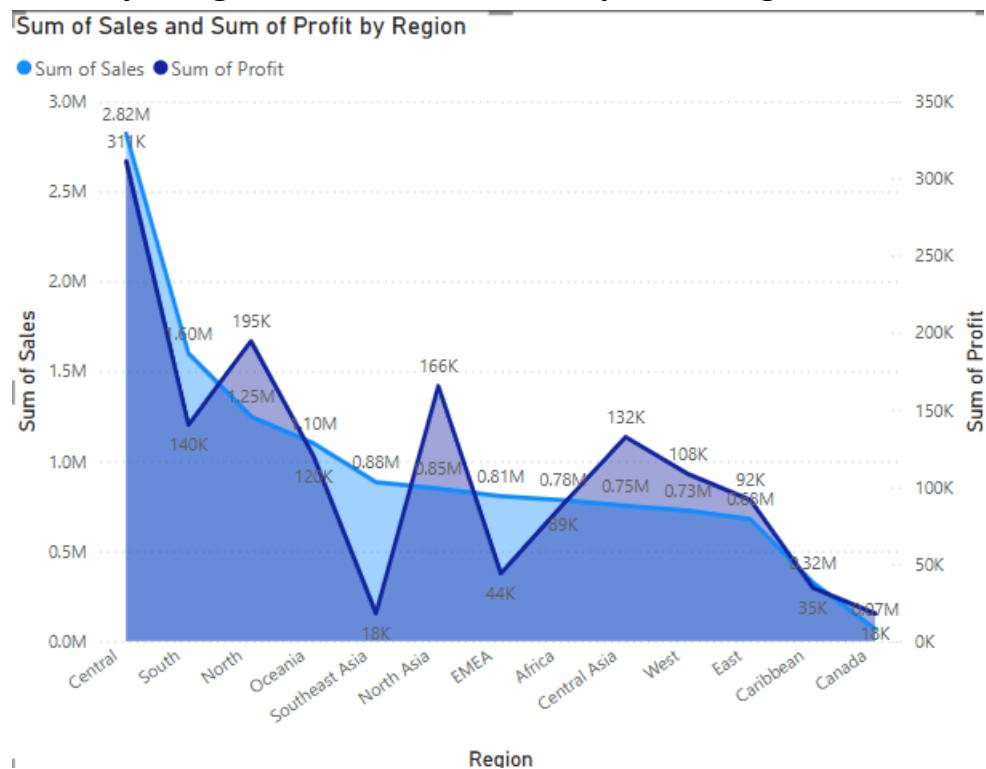
1.Analyze country wise total profit using bubble map chart



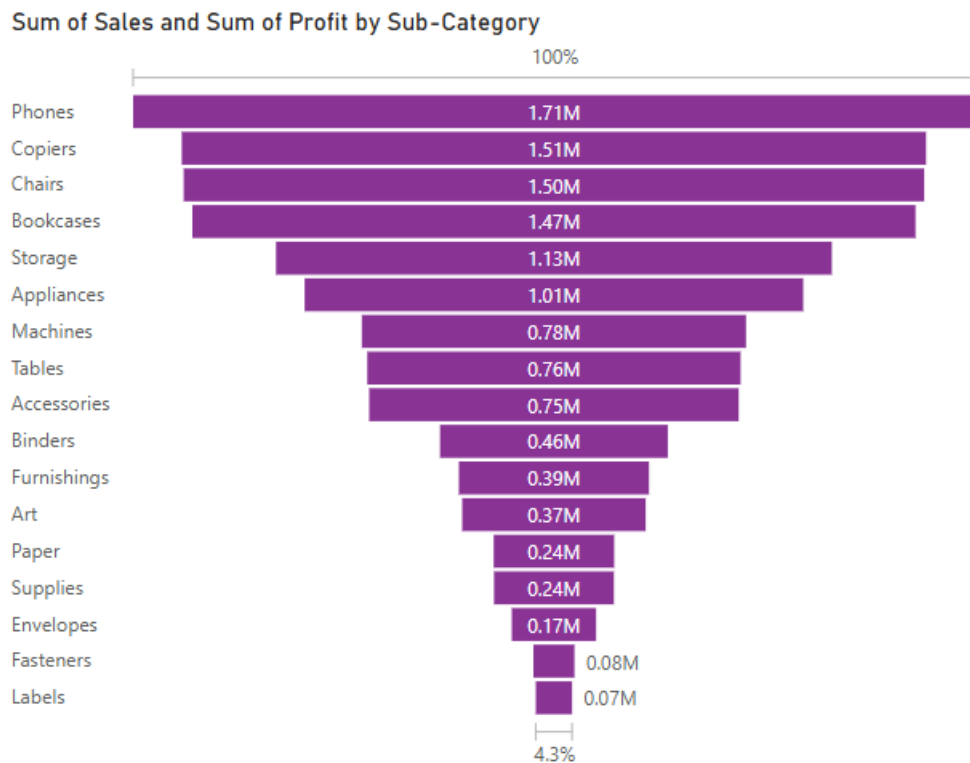
2. Analyze segment wise total sales using clustered column chart



3. Analyze region wise total sales and profit using area chart.

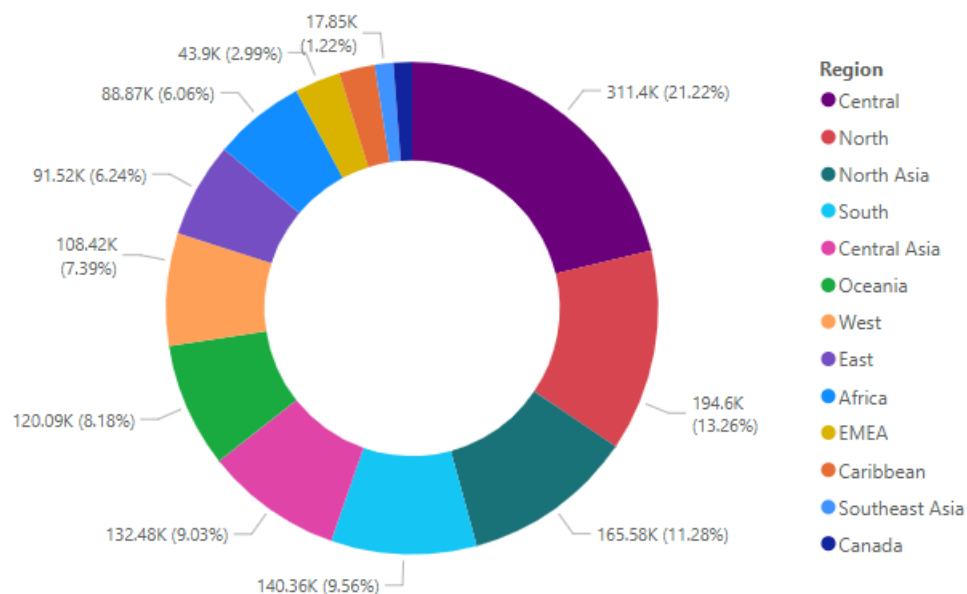


4. Display subcategory wise total sales using funnel chart



5. Display profit by region using donut chart

Sum of Profit by Region



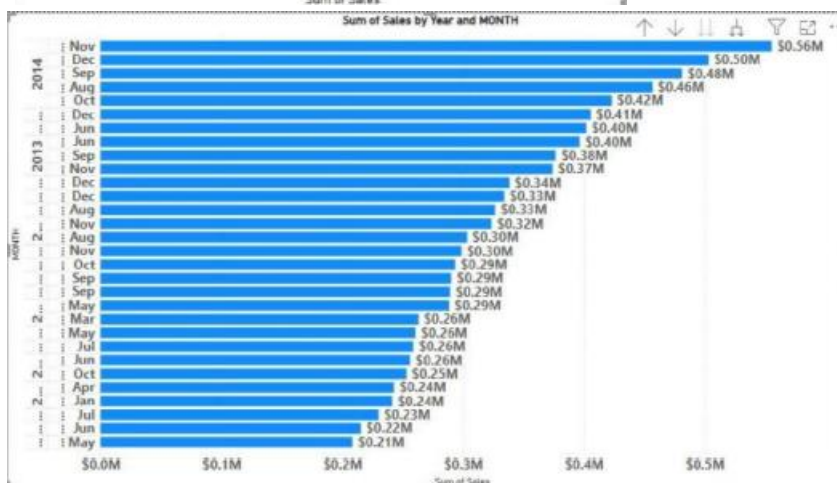
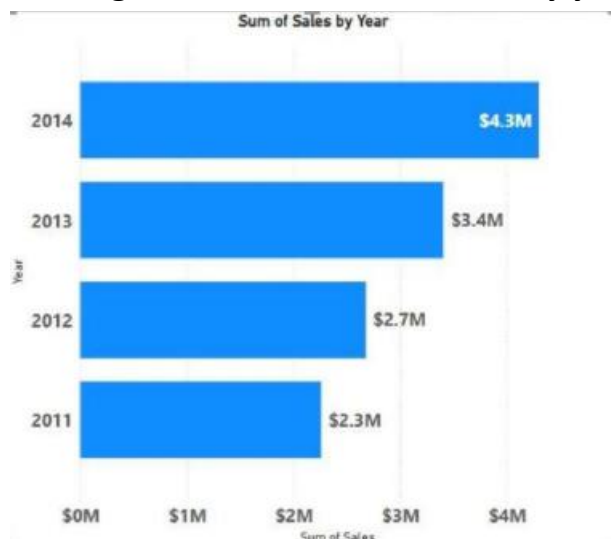
6. Display Sum of quantity using Gauge chart & Insert bar chart to display year wise sum of sales



7. Insert Pivot table sales data month wise for each year and do conditional formatting using rule-based data range color display

Year	Sum of Sales
2011	\$2,259,450.89554
2012	\$2,677,438.694399979
2013	\$3,405,746.449379981
2014	\$4,299,865.870560023
Total	\$12,642,501.90987998

8. Use concept hierarchy and explore drill up and drill down operations by creating bar chart of sum of sales by year.



Conclusion: LO4, LO6 achieved

BI Lab Assignment - 3

Student

student_id

student-name

major

course-section

course_id

section_no

course name

units

room_id

room capacity

Grades

student_id

course_id

prof_id

semester_id

course-grade

(fact table)

Professor

prof_id

prof-name

title

dept_id

dept name

Period

semester_id

year

Q.2

size of the table

- ① student_id : 5 bytes
- ② course_id : 5 bytes
- ③ prof_id : 5 bytes
- ④ semester_id : 5 bytes
- ⑤ course_grade : 5 bytes

Total size per row (Assumption : 25 bytes)

No. of courses : 500

No. of students per course : 60

Time : 30 months

1 sem = 6 months (Assumption)

$$\therefore \text{Total rows} = 500 \times 60 \times 5 = 150000$$

$$\begin{aligned} \text{Size of fact table} &= 150000 \times 25 \\ &= 3750000 \text{ bytes} \end{aligned}$$

course_section

course_id

section_no

course_name

units

room_id

Room

room_id

room capacity

Grades

Student

student_id

course_id

semester_id

course grade

student_id

student_name

major_id

Professor

prof_id

prof name

dept_id

Major

Period

Department

major_id

major name

semester_id

year

dept_id

dept name

course_section

course_id

section_no

course_name

units

room_id

room_capacity

Student

student_id

student_name

major

Attendance

student_id

course_id

semester_id

Period

semester_id

year

BI Lab Assignment 4

Aim:

Q. Create OLAP CuBE using MySQL Server.

Steps:

- ① Open SQL Server Management Studio 2012
- ② Connect Database Engine Click Connect
- ③ Open New Query editor
- ④ Go to extracted SQL file & doubleclick on it
- ⑤ New SQL Query Editor will be opened containing Sales DW Database.
- ⑥ Click on execute or press F5 by selecting query one by one or directly clicking on execute.
- ⑦ After completing execution, Save & close SQL Server Management Studio lamp
- ⑧ Start SSDT (SQL Server Data Tools) environment & create new Data Source.
- ⑨ Go to SQL Server. Click on title → Right click & run as admission. Click on title → New → Project.
- ⑩ In BI → Analysis services & Data mining models → project name → click OK.
- ⑪ Right click on Data sources in solution explores → New Data Source. Data Source wizard appears → Click on New → Select Server Name → Select use SQL Server Authentication → Select or enter a → Database name (Sales-DW)
- ⑫ Click next
- ⑬ Click finish, as Sales-DW gets created
- ⑭ Creating New Data Source View in Solution Explorer: Right click on Data Source View → Select new Data → Source View → click next → click vey → Select Fact Product Sales (dbu) & put to include objects by → clicking on click on Add related tables
- ⑮ Click Next Sales DW.dsv appears in data Source View in solution Explorer.

- ⑩ Creating new cube. Right click on cubes → new cube → select use existing tables in select creation method → next
- ⑪ In select measure Group tables → select Fact ProductSales → Click Next.
- ⑫ In select measures → check all measures → next
- ⑬ In select new dimensions → check all dimensions → next
- ⑭ Click on finish. Sales Dw cube is created.
- ⑮ Dimension modified in dimension tab → double click DimProduct.dim
- ⑯ Drag & drop Product Name from Table in Data Source View & Add in Attribute at left side.
- ⑰ Creating Attribute Hierarchy in Date Dimension. Double click on Dim Date dimension. Drag & drop fields from table in data source view to attributes
- ⑱ Drag & Drop attributes from leftmost pane of attributes to middle pane of hierarchy
- ⑲ Drag fields in sequence from Attributes to hierarchy window (Year, Name, Month, week)
- ⑳ Deploy cube. Right click on Project name → properties. This window appears
- ㉑ Do following changes & click on apply & ok
- ㉒ Right click on project name → Deploy
- ㉓ Deployment Successful
- ㉔ To process cube right click on Sales - Dw.cube → process
- ㉕ Click Run.
- ㉖ Browser the cube for analysis in solution explorer.

Aarya Arban
T11 – 05
Assignment No. 5

Aim: Tutorial on Data Exploration

1. For the given data set

57, 57, 57, 58, 63, 66, 66, 67, 67, 68, 69, 70, 70, 70, 72, 73, 75, 75, 76, 76, 78, 79, 81 find the

- a) Mean
- b) Median
- c) Mode
- d) Midrange
- e) Range
- f) All quartiles
- g) IQR
- h) Draw boxplot for the above dataset. Comment about outliers.

BI Lab Assignment 5

Q.1 a) $\text{Mean} = \frac{1590}{23} = 69.13$

b) Median $\Rightarrow$ Middle value when data is sorted in ascending order
Median = 70

c) Mode = 57, 70 (Appears 3 times)

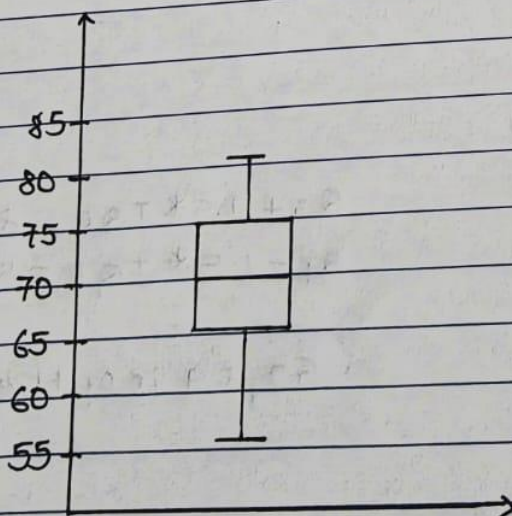
d) $\text{Midrange} = \frac{\text{Min} + \text{Max}}{2} = \frac{57 + 81}{2} = 69$

e) $\text{Range} = 81 - 57 = 24$

f) All quartiles:
 $Q_1 = 66, Q_2 = 70, Q_3 = 75$

g) $\text{IQR} = Q_3 - Q_1$
 $= 75 - 66 = 9$

h) Boxplot:



No outliers

2. 52, 57, 57, 58, 63, 66, 66, 67, 67, 68, 69, 70, 70, 70, 72, 73, 75, 75, 76, 76, 78, 79, 89 find the

a) Mean

b) Median

c) Mode

d) Midrange

e) Range

f) All quartiles

g) IQR

h) Draw boxplot for the above dataset.

Comment about outliers.

Q.2

a) Mean = $\frac{1595}{23} = 69.26$

b) Median = 70

c) Mode = 70

d) Mid range = $\frac{52 + 89}{2} = 70.5$

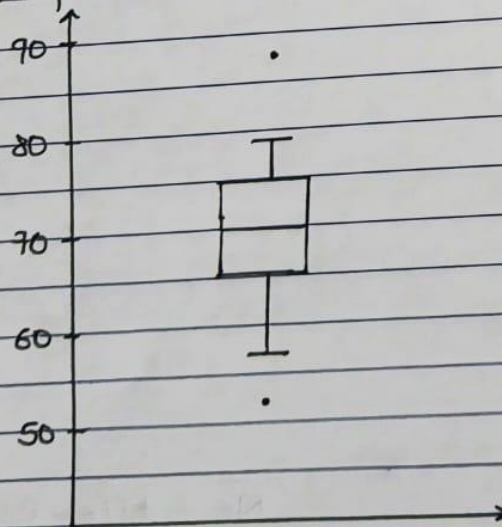
e) Range = $89 - 52 = 37$

f) All quartiles:

$Q_1 = 66, Q_2 = 70, Q_3 = 75$

g) IQR = $75 - 66 = 9$

h) Boxplot:



$Q_3 + 1.5 * IQR = 88.5$

$Q_1 - 1.5 * IQR = 52.5$

$\therefore 52, 89$ is outlier

3. Use MS-Excel to

a. Extract summary of descriptive statistics

File Home Insert Page Layout Formulas Data Review View								
H28		fx						
	A	B	C	D	E	F	G	H
1	Column1							
2								
3	Mean	29.962963						
4	Standard Error	2.49071294						
5	Median	25						
6	Mode	25						
7	Standard Deviation	12.9421241						
8	Sample Variance	167.498575						
9	Kurtosis	2.14199779						
10	Skewness	1.24575654						
11	Range	57						
12	Minimum	13						
13	Maximum	70						
14	Sum	809						
15	Count	27						
16								

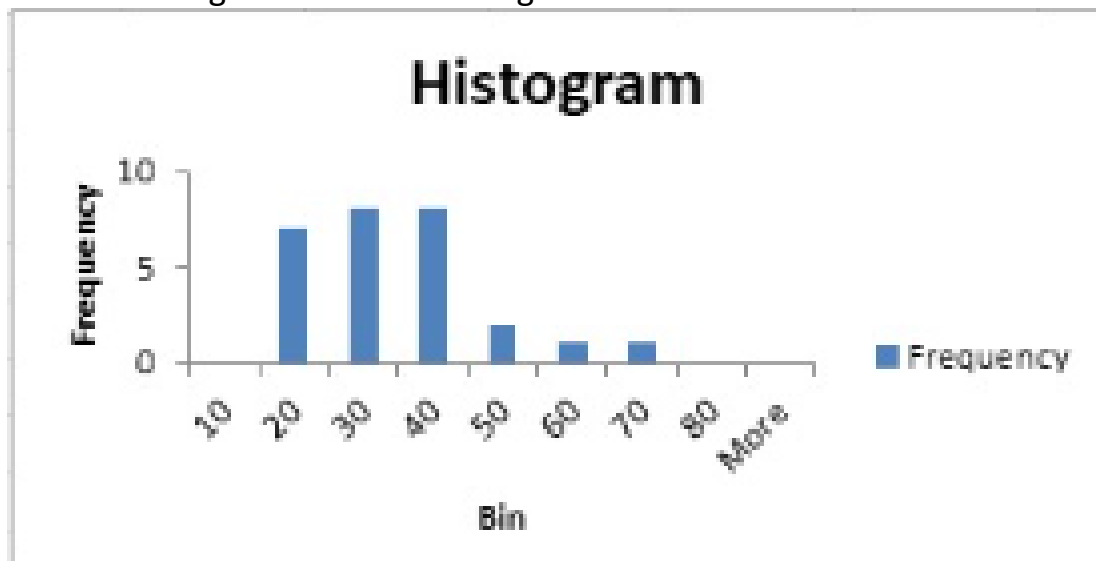
b. percentiles of 10, 20,25, 50,75, 80,90,100

File Home Insert Page Layout Formulas Data Review View								
K22		fx						
	A	B	C	D	E	F	G	H
1	Dataset	Percentile						
2	13	16						
3	15	20						
4	16	20.5						
5	16	25						
6	19	35						
7	20	35.8						
8	20	45.4						
9	21	70						
10	22							
11	22							
12	25							
13	25							
14	25							
15	25							
16	30							
17	33							
18	33							
19	35							
20	35							
21	35							
22	35							
23	36							
24	40							
25	45							
26	46							
27	52							
28	70							
29								
30								

c. Create frequency bin

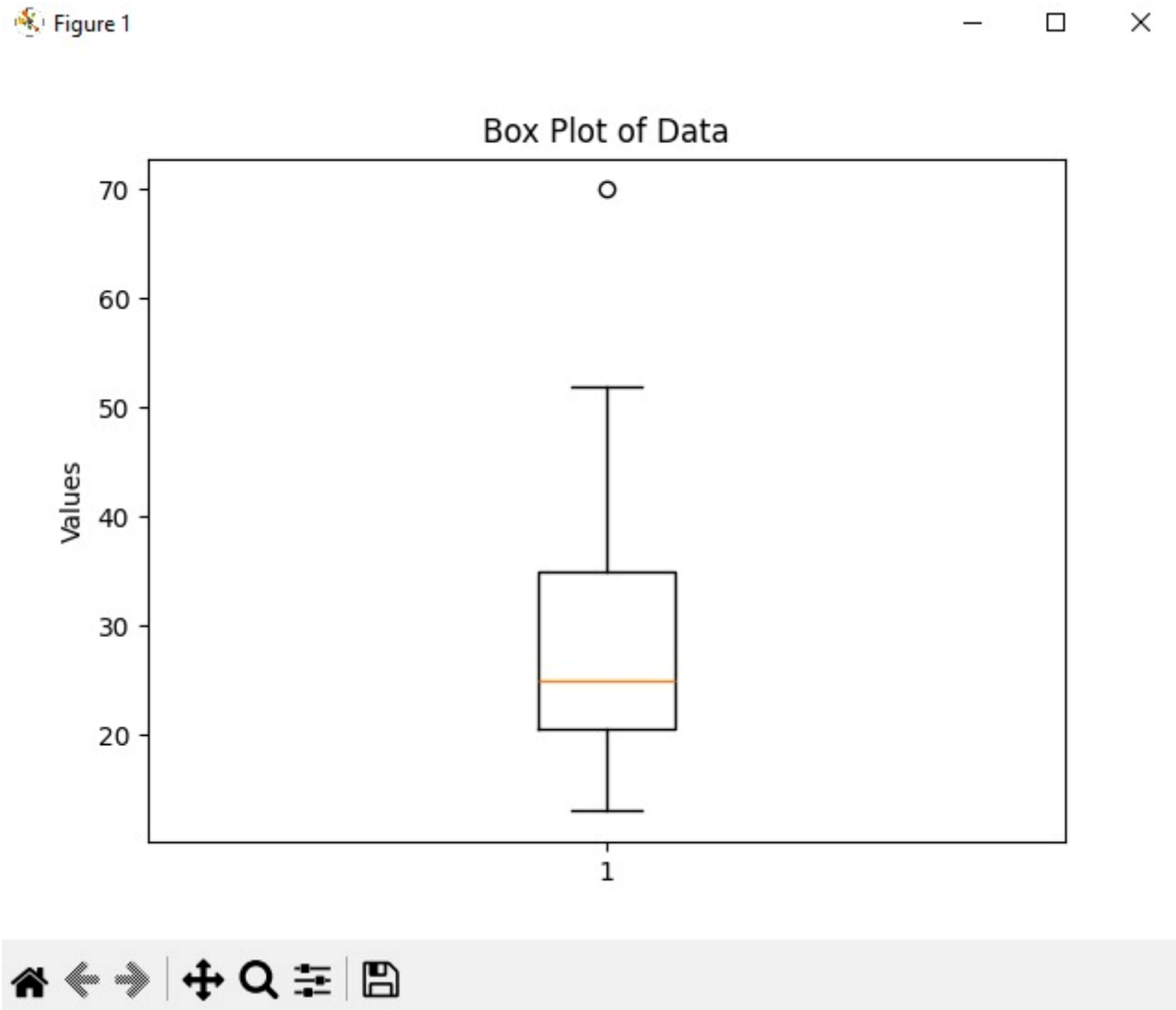
N19			
	A	B	C
1	Bin	Frequency	
2	10	0	
3	20	7	
4	30	8	
5	40	8	
6	50	2	
7	60	1	
8	70	1	
9	80	0	
10	More	0	
11			
12			

d. Draw histogram of the following dataset



e. Draw boxplot of the following dataset

13,15,16,16,19,20,20,21,22,22,25,25,25,25,30,33,33,35,35,35,35,36,40,45,46,52,70



4. Find median of grouped data

CI	F
0-10	4
10-20	6
20-30	18
30-40	34
40-50	40

Q.4

→

C I	F	CF
0-10	4	4
10-20	6	4+6=10
20-30	18	10+18=28
30-40	34	28+34=62
40-50	40	62+40=102

$$\therefore N = 102$$

$$N = 102 = \frac{51}{2}$$

Here CF just exceeding 51 is 62,

$\therefore$ Median class: 30-40

$$\therefore \text{Median} = L + \left(\frac{N/2 - CF}{F} \right) \times h$$

$$= 30 + \left(\frac{51 - 28}{34} \right) \times 10$$

$$= 36.76$$

5. Partition data into 4 bins using equi depth binning method and perform smoothing according to following methods

- a) Smoothing by bin median
- b) Smoothing by bin mean
- c) Smoothing by bin boundaries

Data 11,13,13,15,15,16,19,20,20,20,21,21,22,23,24,30,40,45,45,45,71,72,73,75

Q.5

→ Sorting & dividing the data set equally into 4 bins:

Bin 1 = 11, 13, 13, 15, 15, 16

Bin 2 = 19, 20, 20, 20, 21, 21

Bin 3 = 22, 23, 24, 30, 40, 45

Bin 4 = 45, 45, 71, 72, 73, 75

a) Smoothing by bin median

Median of bin 1 = $(13+15)/2 = 14$

Median of bin 2 = 20, Median of bin 3 = 27

Median of bin 4 = 71.5

∴ Bin 1 = 14, 14, 14, 14, 14, 14; Bin 2 = 20, 20, 20, 20, 20, 20

Bin 3 = 27, 27, 27, 27, 27, 27; Bin 4 = 71.5, 71.5, 71.5, 71.5, 71.5, 71.5

b) Smoothing by Bin Mean

Bin 1 = Mean = 13.83, Bin 2 = Mean = 20.16

Bin 3 = Mean = 30.66, Bin 4 = Mean = 63.5

∴ Bin 1 = 13.83, 13.83, 13.83, 13.83, 13.83, 13.83

Bin 2 = 20.16, 20.16, 20.16, 20.16, 20.16, 20.16

Bin 3 = 30.66, 30.66, 30.66, 30.66, 30.66, 30.66

Bin 4 = 63.5, 63.5, 63.5, 63.5, 63.5, 63.5

c) Smoothing by bin boundaries

Extreme data points in bin are kept the same.

Data points in between are replaced by boundary values which they are close to.

Bin 1 = 11, 11, 11, 16, 16, 16

Bin 2 = 19, 19, 19, 19, 21, 21

Bin 3 = 22, 22, 22, 22, 45, 45

Bin 4 = 45, 45, 75, 75, 75, 75

Aarya Arban

T11 – 05

Assignment No. 6

Aim: Creating Decision Tree using Python

Program:

```
import pandas as pd
```

```
import numpy as np
```

```
from math import log2
```

```
df = pd.read_csv('Buy_Computer.csv')
```

```
def entropy(df, target_column):
```

```
    value_counts = df[target_column].value_counts()
```

```
    total_records = len(df)
```

```
    entropy_value = 0
```

```
    for count in value_counts:
```

```
        prob = count / total_records
```

```
        entropy_value -= prob * log2(prob)
```

```
    return entropy_value
```

```
def information_gain(df, attribute, target_column):
```

```
    total_entropy = entropy(df, target_column)
```

```
    attribute_values = df[attribute].value_counts()
```

```
    weighted_entropy = 0
```

```
    for value, count in attribute_values.items():
```

```

    subset = df[df[attribute] == value]

    weighted_entropy += (count / len(df)) * entropy(subset,
target_column)

return total_entropy - weighted_entropy

# First iteration
target_column = 'Buy_Computer'
attributes = ['age', 'income', 'student', 'credit_rating']

# Calculate Information Gain for each attribute
information_gains = {}
for attribute in attributes:
    information_gains[attribute] = information_gain(df, attribute,
target_column)

# Find the best attribute to split on (first iteration split by 'age')
best_attribute = max(information_gains, key=information_gains.get)

# Split based on the best attribute 'age' and print subsets
subsets = {}
for value in df[best_attribute].unique():
    subset = df[df[best_attribute] == value]
    subsets[value] = subset

# Print Information Gain for first iteration (age)
print("Information Gain for each attribute (First Iteration):")

```



```

for attribute, gain in information_gains.items():
    print(f"{attribute}: {gain}")

print(f"\nBest attribute to split on: {best_attribute}")
print("\nThe dataset is split based on the attribute:", best_attribute)

# Now, we will process only 'youth' and 'senior' subsets for the second
iteration

print("\nSecond Iteration for youth and senior subsets:")

for value in ['youth', 'senior']: # Exclude 'middle_age' subset
    subset = subsets[value]

    print(f"\nSecond Iteration for subset where {best_attribute} = {value}:")

    # Exclude 'age' from the remaining attributes (since 'age' was already
    used)

    remaining_attributes = [attr for attr in attributes if attr != best_attribute]

    # Calculate Information Gain for each of the remaining attributes in this
    subset

    information_gains_2 = {}

    for attribute in remaining_attributes:

        information_gains_2[attribute] = information_gain(subset, attribute,
        target_column)

    # Find the best attribute to split on within the current subset

    best_attribute_2 = max(information_gains_2,
        key=information_gains_2.get)

    print(f"\nInformation Gain for each attribute in {value} subset:")

    for attribute, gain in information_gains_2.items():

```

```

    print(f"{attribute}: {gain}")

print(f"\nBest attribute to split on in this subset: {best_attribute_2}")

# Print the rule for splitting
print(f"\nRule for splitting {value} subset: Split by {best_attribute_2}")

# Split the subset based on the best attribute and print the results
print(f"\nThe dataset is split based on the attribute: {best_attribute_2}")
for val in subset[best_attribute_2].unique():
    subset_split = subset[subset[best_attribute_2] == val]
    print(f"\nSubset where {best_attribute_2} = {val}:")
    print(subset_split)

# Display the rule for this subset
    print(f"\nRule for {best_attribute_2} = {val}: If {best_attribute_2} = {val},
then {target_column} = {'yes' if subset_split[target_column].iloc[0] ==
'yes' else 'no'}")

# Automatically recognize if a subset has a pure class
def check_pure_subset(df, target_column):
    return len(df[target_column].unique()) == 1

# Store tree structure
decision_tree = {}

def build_tree(subset, parent_attribute="Root"):

```

```
if check_pure_subset(subset, target_column):  
    decision_tree[parent_attribute] = subset[target_column].iloc[0]  
    return
```

```
remaining_attributes = [attr for attr in attributes if attr not in  
    decision_tree]
```

```
if not remaining_attributes:  
    return
```

```
information_gains_local = {attr: information_gain(subset, attr,  
    target_column) for attr in remaining_attributes}
```

```
best_attr = max(information_gains_local,  
    key=information_gains_local.get)
```

```
decision_tree[parent_attribute] = best_attr
```

```
for val in subset[best_attr].unique():  
    subset_split = subset[subset[best_attr] == val]  
    build_tree(subset_split, f"{best_attr} = {val}")
```

```
# Build the tree
```

```
for value, subset in subsets.items():  
    build_tree(subset, f"{best_attribute} = {value}")
```

```
# Print the decision tree
```

```
print("\nDecision Tree Representation:")  
for key, value in decision_tree.items():  
    print(f"{key} -> {value}")
```


Output:

Information Gain for each attribute (First Iteration):

age: 0.24674981977443933

income: 0.02922256565895487

student: 0.15183550136234159

credit_rating: 0.04812703040826949

Best attribute to split on: age

The dataset is split based on the attribute: age

Second Iteration for youth and senior subsets:

Second Iteration for subset where age = youth:

Information Gain for each attribute in youth subset:

income: 0.5709505944546686

student: 0.9709505944546686

credit_rating: 0.01997309402197489

Best attribute to split on in this subset: student

Rule for splitting youth subset: Split by student

The dataset is split based on the attribute: student

Subset where student = no:

	id	age	income	student	credit_rating	Buy_Computer
0	1	youth	high	no	fair	no
1	2	youth	high	no	excellent	no
7	8	youth	medium	no	fair	no

Rule for student = no: If student = no, then Buy_Computer = no

Subset where student = yes:

	id	age	income	student	credit_rating	Buy_Computer
8	9	youth	low	yes	fair	yes
10	11	youth	medium	yes	excellent	yes

Rule for student = yes: If student = yes, then Buy_Computer = yes

Second Iteration for subset where age = senior:

Information Gain for each attribute in senior subset:

income: 0.01997309402197489

student: 0.01997309402197489

credit_rating: 0.9709505944546686

Best attribute to split on in this subset: credit_rating

Rule for splitting senior subset: Split by credit_rating

The dataset is split based on the attribute: credit_rating

Subset where credit_rating = fair:

	id	age	income	student	credit_rating	Buy_Computer
3	4	senior	medium	no	fair	yes
4	5	senior	low	yes	fair	yes
9	10	senior	medium	yes	fair	yes

Rule for credit_rating = fair: If credit_rating = fair, then Buy_Computer = yes

Subset where credit_rating = excellent:

	id	age	income	student	credit_rating	Buy_Computer
5	6	senior	low	yes	excellent	no
13	14	senior	medium	no	excellent	no

Rule for credit_rating = excellent: If credit_rating = excellent, then
Buy_Computer = no

Decision Tree Representation:

age = youth -> student

student = no -> no

student = yes -> yes

age = middle -> yes

age = senior -> credit_rating

credit_rating = fair -> yes

credit_rating = excellent -> no


```

        #For each feature value, the conditional probability of the
        feature value given the class label ("yes" or "no") is stored
        self.conditional_prob[i][value] = {
            'yes': count_yes / total_yes if total_yes > 0 else 0,
            'no': count_no / total_no if total_no > 0 else 0
        }
    # predict method takes an instance (a list of feature values) and calculates
    the posterior probability for each class ("yes" and "no")
    def predict(self, instance):
        #prior probability of the class "yes", which is multiplied by the
        conditional probabilities of the instance's feature values given the class "yes".
        prob_instance_yes = self.prob_yes
        prob_instance_no = self.prob_no

        for i, value in enumerate(instance):
            if value in self.conditional_prob[i]:
                prob_instance_yes *= self.conditional_prob[i][value]['yes']
                prob_instance_no *= self.conditional_prob[i][value]['no']

        return 'yes' if prob_instance_yes > prob_instance_no else 'no'

dataset = pd.read_csv('Buy_Computer.csv')

classifier = BayesianClassifier()
classifier.train(dataset)

test_instance = ['middle_age', 'high', 'no', 'fair']
prediction = classifier.predict(test_instance)

print("Predicted class:", prediction)

```

Output:

```

[6] ✓ 0.0s
... Predicted class: yes

```

Aarya Arban

T11 - 05

Assignment No. 8

Aim: Write a program to implement k - means clustering.

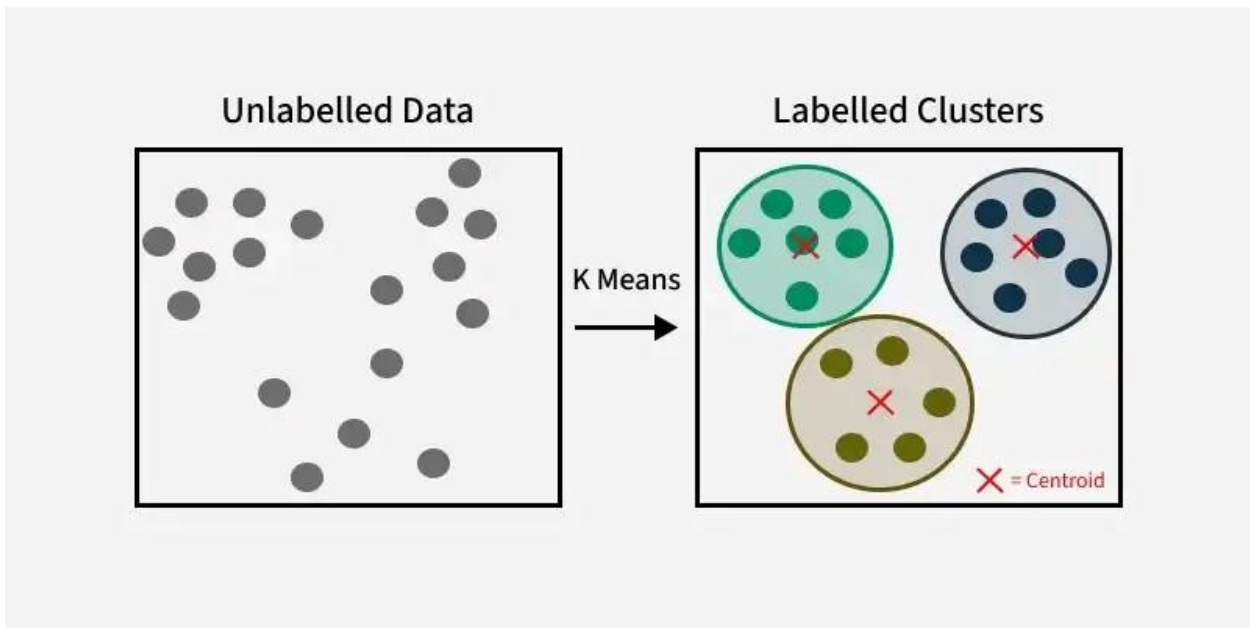
Theory:

K-means clustering is a technique used to organize data into groups based on their similarity. For example online store uses K-Means to group customers based on purchase frequency and spending creating segments like Budget Shoppers, Frequent Buyers and Big Spenders for personalised marketing.

The algorithm works by first randomly picking some central points called centroids and each data point is then assigned to the closest centroid forming a cluster. After all the points are assigned to a cluster the centroids are updated by finding the average position of the points in each cluster. This process repeats until the centroids stop changing forming clusters. The goal of clustering is to divide the data points into clusters so that similar data points belong to same group.

How does k-means clustering work?

We are given a data set of items with certain features and values for these features (like a vector). The task is to categorize those items into groups. To achieve this, we will use the K-means algorithm. 'K' in the name of the algorithm represents the number of groups/clusters we want to classify our items into.



The algorithm will categorize the items into k groups or clusters of similarity. To calculate that similarity, we will use the Euclidean distance as a measurement. The algorithm works as follows:

1. First, we randomly initialize k points, called means or cluster centroids.
2. We categorize each item to its closest mean, and we update the mean's coordinates, which are the averages of the items categorized in that cluster so far.
3. We repeat the process for a given number of iterations and at the end, we have our clusters.

The "points" mentioned above are called means because they are the mean values of the items categorized in them. To initialize these means, we have a lot of options. An intuitive method is to initialize the means at random items in the data set. Another method is to initialize the means at random values between the boundaries of the data set. For example for a feature x the items have values in $[0,3]$ we will initialize the means with values for x at $[0,3]$.

Code:

```
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Example of multidimensional data with 5 features (replace with your actual data)
# Each row represents a data point, and there are 5 features per data point.
data = np.array([
    [1.0, 2.0, 3.0, 4.0, 5.0, 10.0, 11.0, 12.0, 13.0, 14.0],
    [2.0, 3.0, 4.0, 5.0, 6.0, 12.0, 13.0, 14.0, 15.0, 16.0],
    [3.0, 4.0, 5.0, 6.0, 7.0, 12.0, 13.0, 14.0, 15.0, 16.0],
    [10.0, 11.0, 12.0, 13.0, 14.0, 1.0, 2.0, 3.0, 4.0, 5.0],
    [11.0, 12.0, 13.0, 14.0, 15.0, 1.0, 2.0, 3.0, 4.0, 5.0],
    [12.0, 13.0, 14.0, 15.0, 16.0, 2.0, 3.0, 4.0, 5.0, 6.0],
    [18.0, 19.0, 20.0, 21.0, 22.0, 2.0, 3.0, 4.0, 5.0, 6.0],
    [19.0, 20.0, 21.0, 22.0, 23.0, 2.0, 3.0, 4.0, 5.0, 6.0],
    [20.0, 21.0, 22.0, 23.0, 24.0, 2.0, 3.0, 4.0, 5.0, 6.0],
    [30.0, 31.0, 32.0, 33.0, 34.0, 2.0, 3.0, 4.0, 5.0, 6.0]
])

# Function to perform k-means clustering and return the error (inertia)
def kmeans_clustering(data, k):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(data)
    return kmeans.inertia_ # Return the error value (inertia)

# Perform k-means clustering for k=2 and k=3
error_k2 = kmeans_clustering(data, 2)
error_k3 = kmeans_clustering(data, 3)

# Print the error values for both k=2 and k=3
print(f"Error for k=2: {error_k2}")
print(f"Error for k=3: {error_k3}")
```

```
[ ] import numpy as np
    from sklearn.cluster import KMeans
    import matplotlib.pyplot as plt

    # Generate random two-dimensional data with 150 data points
    # Using normal distribution to create scattered data points
    np.random.seed(42) # For reproducibility
    data = np.vstack([
        np.random.normal(loc=[2, 3], scale=[1, 1], size=(50, 2)), # Cluster 1
        np.random.normal(loc=[8, 9], scale=[1, 1], size=(50, 2)), # Cluster 2
        np.random.normal(loc=[14, 15], scale=[1, 1], size=(50, 2)) # Cluster 3
    ])

    # Function to perform k-means clustering and return the error (inertia)
    def kmeans_clustering(data, k):
        kmeans = KMeans(n_clusters=k)
        kmeans.fit(data)
        return kmeans.inertia_ # Return the error value (inertia)

    # Perform k-means clustering for k=2 and k=3
    error_k2 = kmeans_clustering(data, 2)
    error_k3 = kmeans_clustering(data, 3)

    # Print the error values for both k=2 and k=3
    print(f"Error for k=2: {error_k2}")
    print(f"Error for k=3: {error_k3}")

    # Compare the errors to find the ideal k
    if error_k2 < error_k3:
        print("k=2 has the lower error, so it is the ideal choice.")
    else:
        print("k=3 has the lower error, so it is the ideal choice.")
```

```

# Optional: Plot the data and cluster centers for both k=2 and k=3
kmeans_2 = KMeans(n_clusters=2).fit(data)
kmeans_3 = KMeans(n_clusters=3).fit(data)

plt.figure(figsize=(12, 6))

# Plot for k=2
plt.subplot(1, 2, 1)
plt.scatter(data[:, 0], data[:, 1], c=kmeans_2.labels_, cmap='viridis')
plt.title('K-means clustering for k=2')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.scatter(kmeans_2.cluster_centers[:, 0], kmeans_2.cluster_centers[:, 1], marker='x', color='red', s=200, label='Centroids')
plt.legend()

# Plot for k=3
plt.subplot(1, 2, 2)
plt.scatter(data[:, 0], data[:, 1], c=kmeans_3.labels_, cmap='viridis')
plt.title('K-means clustering for k=3')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.scatter(kmeans_3.cluster_centers[:, 0], kmeans_3.cluster_centers[:, 1], marker='x', color='red', s=200, label='Centroids')
plt.legend()

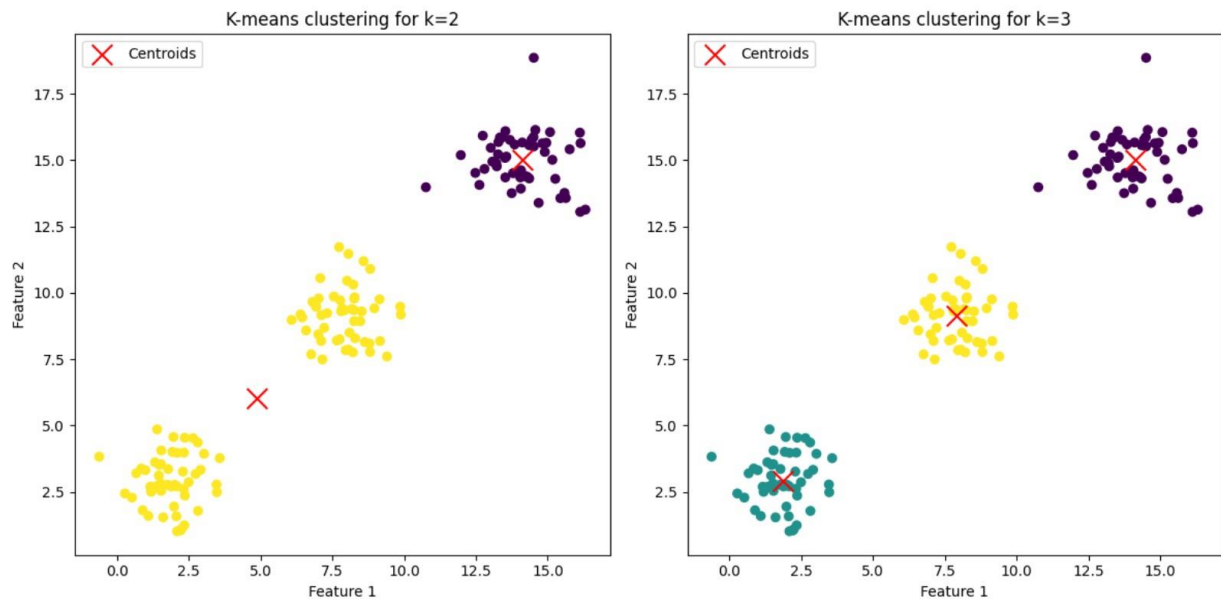
plt.tight_layout()
plt.show()

```

Error for k=2: 2113.4826231862244

Error for k=3: 286.1842142470732

k=3 has the lower error, so it is the ideal choice.



```

import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Generate random two-dimensional data with 150 data points
# Using normal distribution to create scattered data points
np.random.seed(42) # For reproducibility
data = np.vstack([
    np.random.normal(loc=[2, 3], scale=[1, 1], size=(50, 2)), # Cluster 1
    np.random.normal(loc=[8, 9], scale=[1, 1], size=(50, 2)), # Cluster 2
    np.random.normal(loc=[14, 15], scale=[1, 1], size=(50, 2)) # Cluster 3
])

# Function to perform k-means clustering and return the error (inertia)
def kmeans_clustering(data, k):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(data)
    return kmeans

# Perform k-means clustering for k=3
kmeans_3 = kmeans_clustering(data, 3)

# Get the centroids of the clusters
centroids = kmeans_3.cluster_centers_

# Calculate Euclidean distance between centroids
def euclidean_distance(p1, p2):
    return np.sqrt(np.sum((p2 - p1) ** 2))

```



```

# Calculate the distance between all pairs of centroids
distances = {}
for i in range(len(centroids)):
    for j in range(i + 1, len(centroids)):
        distance = euclidean_distance(centroids[i], centroids[j])
        distances[f"Centroid {i+1} to Centroid {j+1}"] = distance

# Print distances between the centroids
for pair, distance in distances.items():
    print(f"Distance between {pair}: {distance:.2f}")

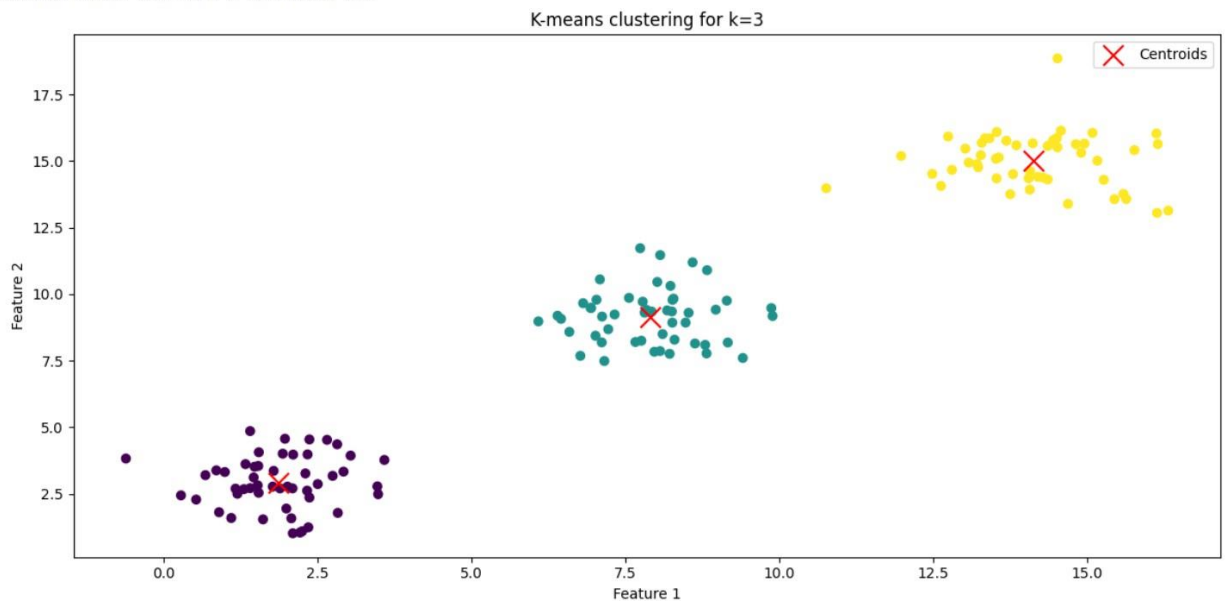
# Optional: Plot the data and cluster centers for k=3
plt.figure(figsize=(12, 6))

# Plot for k=3
plt.scatter(data[:, 0], data[:, 1], c=kmeans_3.labels_, cmap='viridis')
plt.title('K-means clustering for k=3')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.scatter(centroids[:, 0], centroids[:, 1], marker='x', color='red', s=200, label='Centroids')
plt.legend()

plt.tight_layout()
plt.show()

```

Distance between Centroid 1 to Centroid 2: 8.66
 Distance between Centroid 1 to Centroid 3: 17.21
 Distance between Centroid 2 to Centroid 3: 8.55



```

import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Generate random two-dimensional data with 150 data points
# Using normal distribution to create scattered data points
np.random.seed(42) # For reproducibility
data = np.vstack([
    np.random.normal(loc=[2, 3], scale=[1, 1], size=(50, 2)), # Cluster 1
    np.random.normal(loc=[8, 9], scale=[1, 1], size=(50, 2)), # Cluster 2
    np.random.normal(loc=[14, 15], scale=[1, 1], size=(50, 2)) # Cluster 3
])

# Function to perform k-means clustering and return the error (inertia)
def kmeans_clustering(data, k):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(data)
    return kmeans

# Perform k-means clustering for k=2
kmeans_2 = kmeans_clustering(data, 2)

# Get the centroids of the clusters
centroids_2 = kmeans_2.cluster_centers_

# Calculate Euclidean distance between centroids
def euclidean_distance(p1, p2):
    return np.sqrt(np.sum((p2 - p1) ** 2))

```

```

# Calculate the distance between the centroids for k=2
distances_2 = {}
for i in range(len(centroids_2)):
    for j in range(i + 1, len(centroids_2)):
        distance = euclidean_distance(centroids_2[i], centroids_2[j])
        distances_2[f"Centroid {i+1} to Centroid {j+1}"] = distance

# Print distances between the centroids for k=2
for pair, distance in distances_2.items():
    print(f"Distance between {pair}: {distance:.2f}")

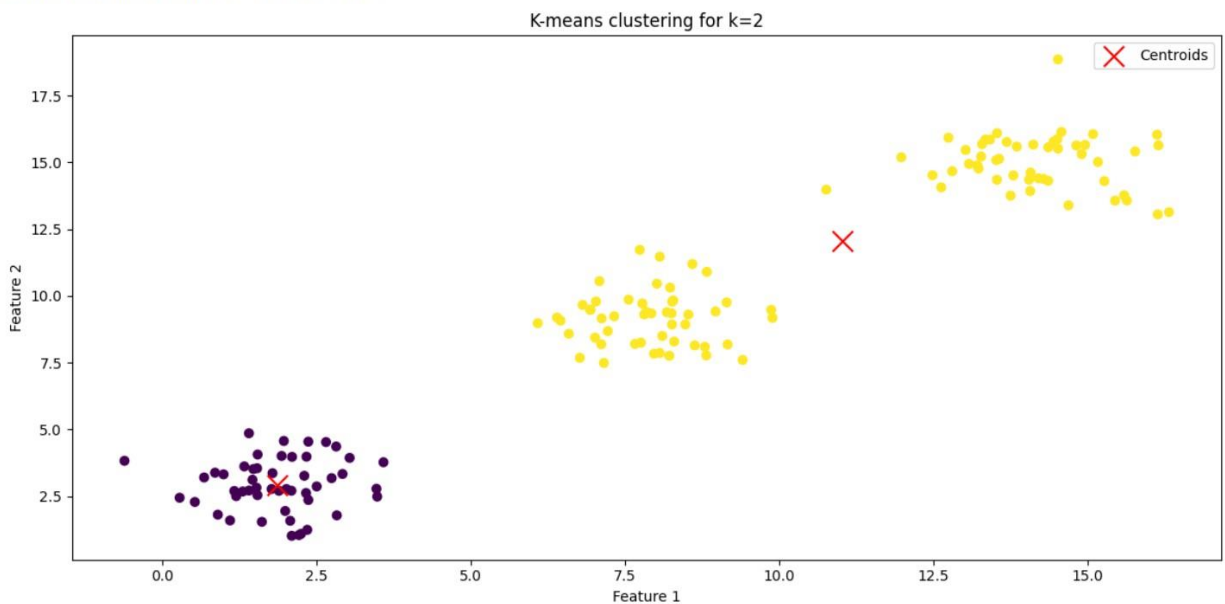
# Optional: Plot the data and cluster centers for k=2
plt.figure(figsize=(12, 6))

# Plot for k=2
plt.scatter(data[:, 0], data[:, 1], c=kmeans_2.labels_, cmap='viridis')
plt.title('K-means clustering for k=2')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.scatter(centroids_2[:, 0], centroids_2[:, 1], marker='x', color='red', s=200, label='Centroids')
plt.legend()

plt.tight_layout()
plt.show()

```

Distance between Centroid 1 to Centroid 2: 12.94



Conclusion:

The K-Means clustering algorithm successfully grouped the data, but choosing **k=2** for a dataset with three natural clusters led to suboptimal results. The computed centroids and their distances indicate how the algorithm merged clusters. Visualization shows the grouping, but selecting the right **k** using methods like the **Elbow Method** or **Silhouette Score** is crucial for better accuracy.

Aarya Arban

T11 – 05

Assignment No. 9

Weka (Waikato Environment for Knowledge Analysis) is a popular, open-source data mining and machine learning software developed at the University of Waikato in New Zealand. It is written in Java and provides a collection of visualization tools and algorithms for data analysis and predictive modeling, along with graphical user interfaces for easy access to these functions.

How Weka is Used in Data Mining

Weka is widely used in data mining tasks such as:

1. Data Preprocessing

- Cleaning data, removing duplicates, normalization, etc.
- Supports different data formats like CSV and ARFF.

2. Classification

- Apply machine learning algorithms like Decision Trees (J48), Naive Bayes, k-NN, SVM, etc.

3. Clustering

- Group similar instances using algorithms like k-Means, DBSCAN, etc.

4. Association Rule Mining

- Discover interesting relations using algorithms like Apriori.

5. Regression

- Perform linear and non-linear regression analysis.

6. Visualization

- Visualize datasets, decision trees, clusters, and evaluation metrics.

7. Evaluation

- Cross-validation, confusion matrix, ROC curves, and other performance metrics.

Comparison of Weka and Other Data Mining Tools

1. Weka

- **Language:** Java
 - **Interface:** GUI + Command Line
 - **Ease of Use:** Very beginner-friendly
 - **Best For:** Education, prototyping ML models
 - **Features:** Classification, clustering, association, regression, visualization
 - **Limitation:** Not ideal for big data or deep learning
-

2. RapidMiner

- **Language:** Java
 - **Interface:** GUI-based drag-and-drop
 - **Ease of Use:** Very user-friendly, no coding required
 - **Best For:** Business analytics, fast prototyping
 - **Features:** ML, data prep, visualization, deployment
 - **Limitation:** Limited free version, commercial license for full features
-

3. KNIME

- **Language:** Java
- **Interface:** Visual workflow GUI
- **Ease of Use:** Easy to learn for non-programmers
- **Best For:** Data integration + machine learning
- **Features:** Data cleaning, analytics, reporting, ML
- **Limitation:** Limited advanced deep learning capabilities out-of-the-box

4. Orange

- **Language:** Python
 - **Interface:** GUI + Python scripting
 - **Ease of Use:** Highly intuitive visual programming
 - **Best For:** Education, visual data exploration
 - **Features:** Classification, clustering, text mining, visualization
 - **Limitation:** Less support for big data or complex pipelines
-

5. R / RStudio

- **Language:** R
 - **Interface:** Code-based (IDE)
 - **Ease of Use:** Steeper learning curve
 - **Best For:** Statistical analysis, academic research
 - **Features:** Extensive ML packages, data visualization
 - **Limitation:** Needs programming knowledge
-

6. Python (Scikit-learn, Pandas, TensorFlow, etc.)

- **Language:** Python
 - **Interface:** Code-based (Jupyter, IDE)
 - **Ease of Use:** Great for programmers
 - **Best For:** Custom ML workflows, scalability
 - **Features:** ML, DL, big data, visualization, automation
 - **Limitation:** Requires coding and understanding of ML concepts
-

7. SAS Enterprise Miner

- **Language:** SAS
 - **Interface:** GUI
 - **Ease of Use:** Designed for business users
 - **Best For:** Enterprise-level analytics
 - **Features:** Predictive modeling, data prep, scoring
 - **Limitation:** Expensive, limited to SAS ecosystem
-

8. IBM SPSS Modeler

- **Language:** Proprietary (IBM)
 - **Interface:** GUI
 - **Ease of Use:** Drag-and-drop interface
 - **Best For:** Marketing, finance, CRM analytics
 - **Features:** Advanced analytics, ML, data prep
 - **Limitation:** Commercial license required
-

9. H2O.ai

- **Language:** Java, R, Python APIs
- **Interface:** GUI + Code (Flow, Python, R)
- **Ease of Use:** Developer and enterprise friendly
- **Best For:** Scalable machine learning and AI
- **Features:** AutoML, big data support, cloud-native
- **Limitation:** Complex for beginners

Implementation of Classification Algorithm (Naïve Bayes) in WEKA:

The WEKA Explorer interface shows the 'Buy_Computer' dataset. The 'Attributes' list on the left includes 'id', 'age', 'income', 'student', 'credit_rating', and 'Buy_Computer'. The 'Selected attribute' table on the right shows the distribution of the 'Buy_Computer' attribute.

No.	Label	Count	Weight
1	no	5	5
2	yes	9	9

Class: Buy_Computer (Nom) Visualize All

The visualization shows two bars: a blue bar for 'no' (count 5) and a red bar for 'yes' (count 9).

Output:

The WEKA Explorer interface shows the 'NaiveBayes' classifier output. The 'Test options' section shows 'Percentage split' set to 70%. The 'Classifier output' section displays the results of the classification.

```
=== Run information ===
Scheme: weka.classifiers.bayes.NaiveBayes
Relation: Buy_Computer
Instances: 14
Attributes: 6
id
age
income
student
credit_rating
Buy_Computer
Test mode: split 70.0% train, remainder test

=== Classifier model (full training set) ===

Naive Bayes Classifier

Attribute Class no yes
(0.38) (0.63)

=====
id
mean 6.2 8.2222
std. dev. 4.6648 3.4247
weight sum 5 9
precision 1 1

age
youth 4.0 3.0
middle_age 1.0 5.0
senior 3.0 4.0
[total] 8.0 12.0

income
high 3.0 3.0
medium 3.0 5.0
low 1.0 1.0
```

```

income
  high      3.0   3.0
  medium    3.0   5.0
  low       2.0   4.0
  [total]   8.0  12.0

student
  no        5.0   4.0
  yes       2.0   7.0
  [total]   7.0  11.0

credit_rating
  fair      3.0   7.0
  excellent 4.0   4.0
  [total]   7.0  11.0

```

Time taken to build model: 0 seconds

=== Evaluation on test split ===

Time taken to test model on test split: 0 seconds

=== Summary ===

```

Correctly Classified Instances      2          50   %
Incorrectly Classified Instances    2          50   %
Kappa statistic                     0
Mean absolute error                 0.4652
Root mean squared error             0.5224
Relative absolute error             93.0465 %
Root relative squared error         99.1171 %
Total Number of Instances          4

```

=== Detailed Accuracy By Class ===

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
	0.000	0.000	?	0.000	?	?	0.750	0.833	no
	1.000	1.000	0.500	1.000	0.667	?	0.750	0.833	yes
Weighted Avg.	0.500	0.500	?	0.500	?	?	0.750	0.833	

=== Confusion Matrix ===

```

a b  <-- classified as
0 2 | a = no
0 2 | b = yes

```

Implementation of Association in WEKA:

The screenshot shows the WEKA Explorer interface with the 'Associate' tab selected. The 'Apriori' algorithm is chosen, and the 'Result list' shows the output for the '145938 - Apriori' task.

Apriori output

```

=== Run information ===

Scheme:      weka.associations.Apriori -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1
Relation:    buys_computer
Instances:    14
Attributes:   5
  age
  income
  student
  credit_rating
  buys_computer

=== Apriori model (full training set) ===

Apriori
=====

Minimum support: 0.15 (2 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 17

Generated sets of large itemsets:

Size of set of large itemsets L(1): 12
Size of set of large itemsets L(2): 47
Size of set of large itemsets L(3): 39
Size of set of large itemsets L(4): 6

Best rules found:

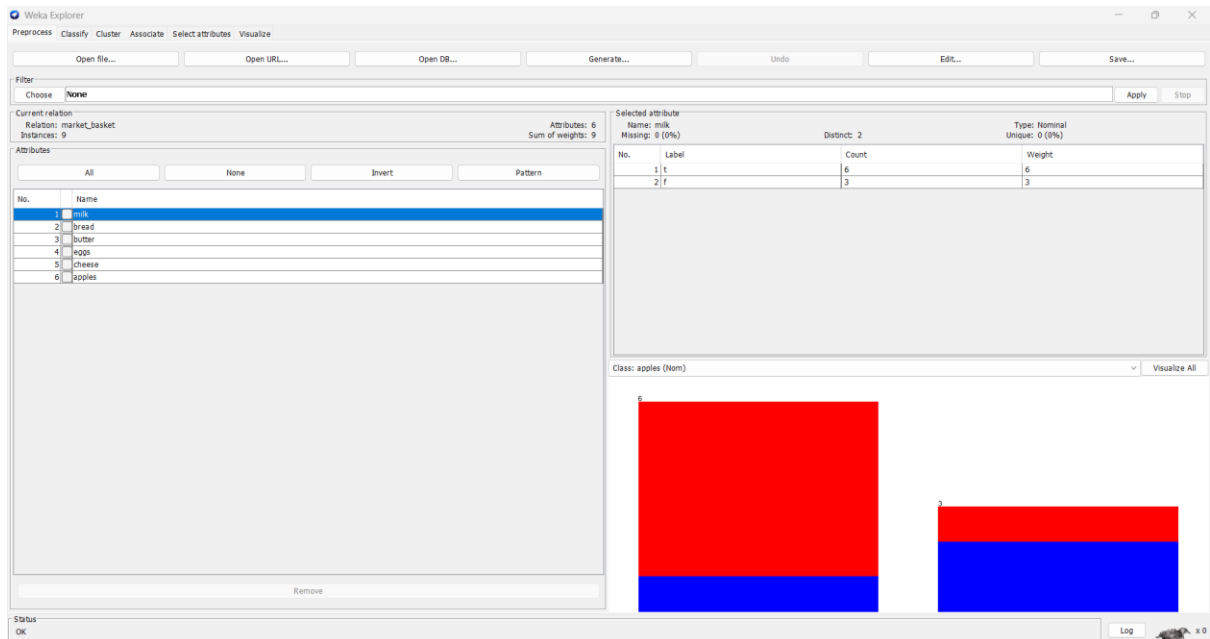
1. age=middle_aged 4 ==> buys_computer=yes 4   <conf:(1)> lift:(1.56) lev:(0.1) [1] conv:(1.43)
2. income=low 4 ==> student=yes 4   <conf:(1)> lift:(2) lev:(0.14) [2] conv:(2)
3. student=yes credit_rating=fair 4 ==> buys_computer=yes 4   <conf:(1)> lift:(1.56) lev:(0.1) [1] conv:(1.43)
4. age=youth buys_computer=no 3 ==> student=no 3   <conf:(1)> lift:(2) lev:(0.11) [1] conv:(1.5)
5. income=middle_aged 3 ==> buys_computer=no 3   <conf:(1)> lift:(2) lev:(0.11) [1] conv:(1.5)

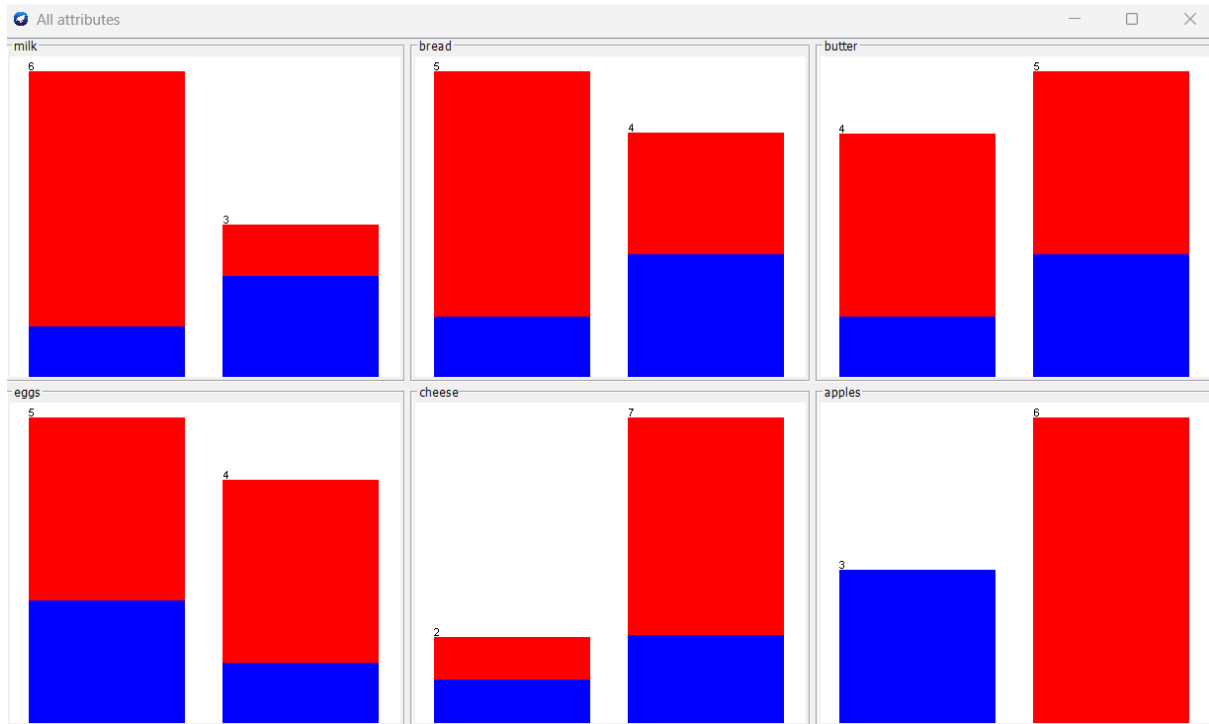
```

Best rules found:

1. age=middle_aged 4 ==> buys_computer=yes 4 <conf:(1)> lift:(1.56) lev:(0.1) [1] conv:(1.43)
2. income=low 4 ==> student=yes 4 <conf:(1)> lift:(2) lev:(0.14) [2] conv:(2)
3. student=yes credit_rating=fair 4 ==> buys_computer=yes 4 <conf:(1)> lift:(1.56) lev:(0.1) [1] conv:(1.43)
4. age=youth buys_computer=no 3 ==> student=no 3 <conf:(1)> lift:(2) lev:(0.11) [1] conv:(1.5)
5. age=youth student=no 3 ==> buys_computer=no 3 <conf:(1)> lift:(2.8) lev:(0.14) [1] conv:(1.93)
6. age=senior buys_computer=yes 3 ==> credit_rating=fair 3 <conf:(1)> lift:(1.75) lev:(0.09) [1] conv:(1.29)
7. age=senior credit_rating=fair 3 ==> buys_computer=yes 3 <conf:(1)> lift:(1.56) lev:(0.08) [1] conv:(1.07)
8. income=low buys_computer=yes 3 ==> student=yes 3 <conf:(1)> lift:(2) lev:(0.11) [1] conv:(1.5)
9. age=youth income=high 2 ==> student=no 2 <conf:(1)> lift:(2) lev:(0.07) [1] conv:(1)
10. income=high buys_computer=no 2 ==> age=youth 2 <conf:(1)> lift:(2.8) lev:(0.09) [1] conv:(1.29)

Implementations of Apriori algorithm using WEKA for 9 transactions dataset:





Output:

```

Weka Explorer
Preprocess  Classify  Cluster  Associate  Select attributes  Visualize
Associate
Choose  Apriori -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1

Start  Stop
Result list (right-click for details)
1. OK: Apriori

Associate output
=== Run information ===
Scheme:      weka.associations.Apriori -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1
Relation:    market_basket
Instances:    9
Attributes:   6
    milk
    bread
    butter
    eggs
    cheese
    apples

=== Associator model (full training set) ===

Apriori
=====
Minimum support: 0.35 (3 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 13

Generated sets of large itemsets:
Size of set of large itemsets L(1): 11
Size of set of large itemsets L(2): 23
Size of set of large itemsets L(3): 17
Size of set of large itemsets L(4): 5

Best rules found:
1. bread=t 5 ==> cheese=f 5   <conf:(1)> lift:(1.29) lev:(0.12) [1] conv:(1.11)
2. eggs=f 4 ==> milk=t 4     <conf:(1)> lift:(1.5) lev:(0.15) [1] conv:(1.33)
3. butter=t 4 ==> cheese=f 4   <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
4. eggs=f 4 ==> cheese=f 4     <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
  
```

Best rules found:

1. bread=t 5 ==> cheese=f 5 <conf:(1)> lift:(1.29) lev:(0.12) [1] conv:(1.11)
2. eggs=f 4 ==> milk=t 4 <conf:(1)> lift:(1.5) lev:(0.15) [1] conv:(1.33)
3. butter=t 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
4. eggs=f 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
5. milk=t bread=t 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
6. eggs=f cheese=f 4 ==> milk=t 4 <conf:(1)> lift:(1.5) lev:(0.15) [1] conv:(1.33)
7. milk=t eggs=f 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
8. eggs=f 4 ==> milk=t cheese=f 4 <conf:(1)> lift:(1.8) lev:(0.2) [1] conv:(1.78)
9. bread=t apples=f 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
10. milk=f 3 ==> eggs=t 3 <conf:(1)> lift:(1.8) lev:(0.15) [1] conv:(1.33)

Sales Analysis and Business Strategy Optimization

GROUP MEMBERS:

- Aarya Arban (05)
- Soham Babsheyte (06)
- Aarna Bafna (07)
- Dhruv Bharwada (08)

INTRODUCTION

This project aims to analyze sales data to uncover key trends, customer behavior, and performance patterns across products, regions, and time. By combining Exploratory Data Analysis (EDA), clustering, forecasting with Facebook Prophet, and visualization techniques, we derive actionable insights to support data-driven decision-making and business growth.

DATASET AND FEATURES

```
[3] sales_dataset.head()
```

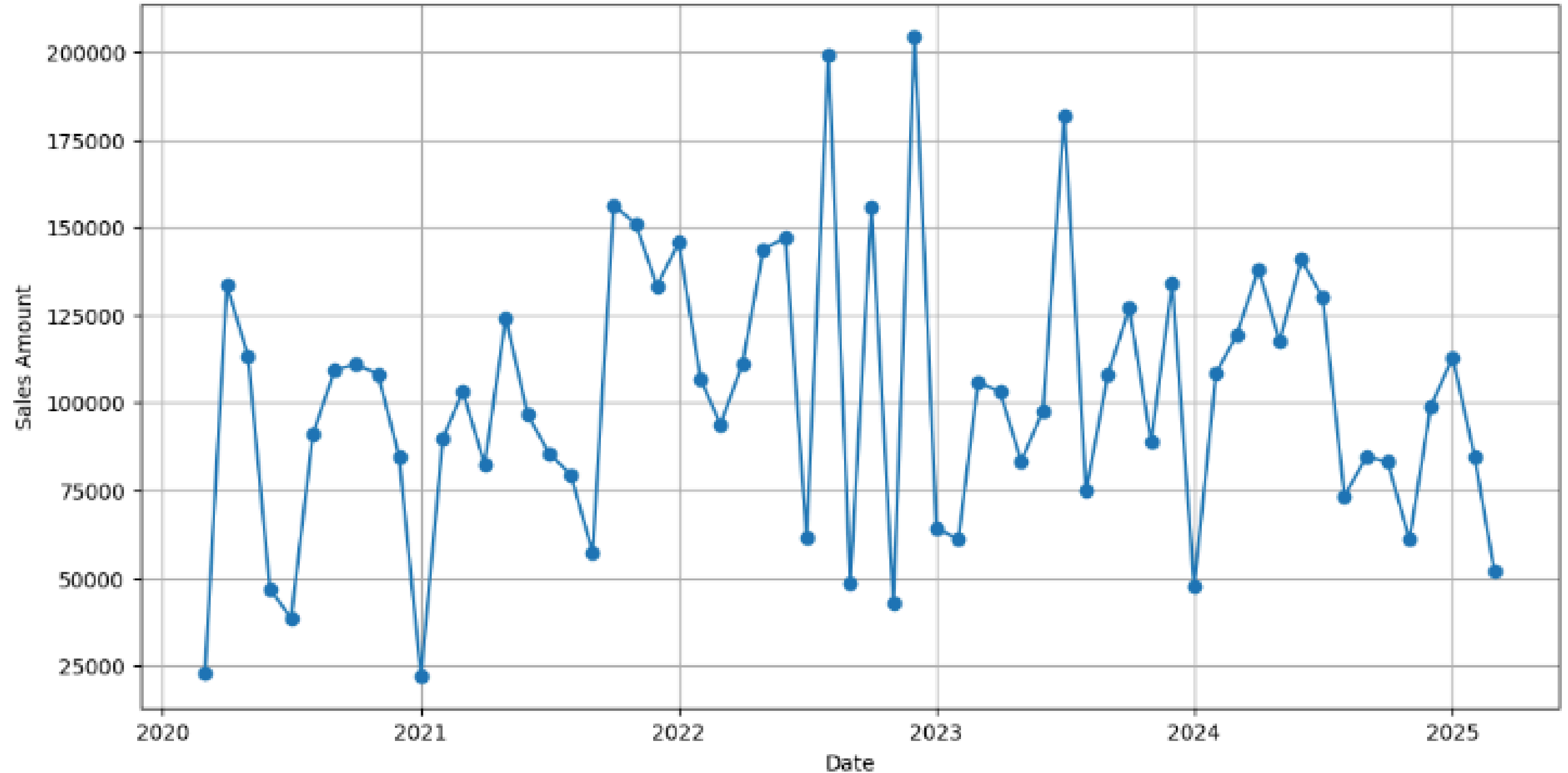
	Order ID	Amount	Profit	Quantity	Category	Sub-Category	PaymentMode	Order Date	CustomerName	State	City	Year-Month
0	B-26776	9726	1275	5	Electronics	Electronic Games	UPI	2023-06-27	David Padilla	Florida	Miami	2023-06
1	B-26776	9726	1275	5	Electronics	Electronic Games	UPI	2024-12-27	Connor Morgan	Illinois	Chicago	2024-12
2	B-26776	9726	1275	5	Electronics	Electronic Games	UPI	2021-07-25	Robert Stone	New York	Buffalo	2021-07
3	B-26776	4975	1330	14	Electronics	Printers	UPI	2023-06-27	David Padilla	Florida	Miami	2023-06
4	B-26776	4975	1330	14	Electronics	Printers	UPI	2024-12-27	Connor Morgan	Illinois	Chicago	2024-12

```
sales_dataset.describe()
```

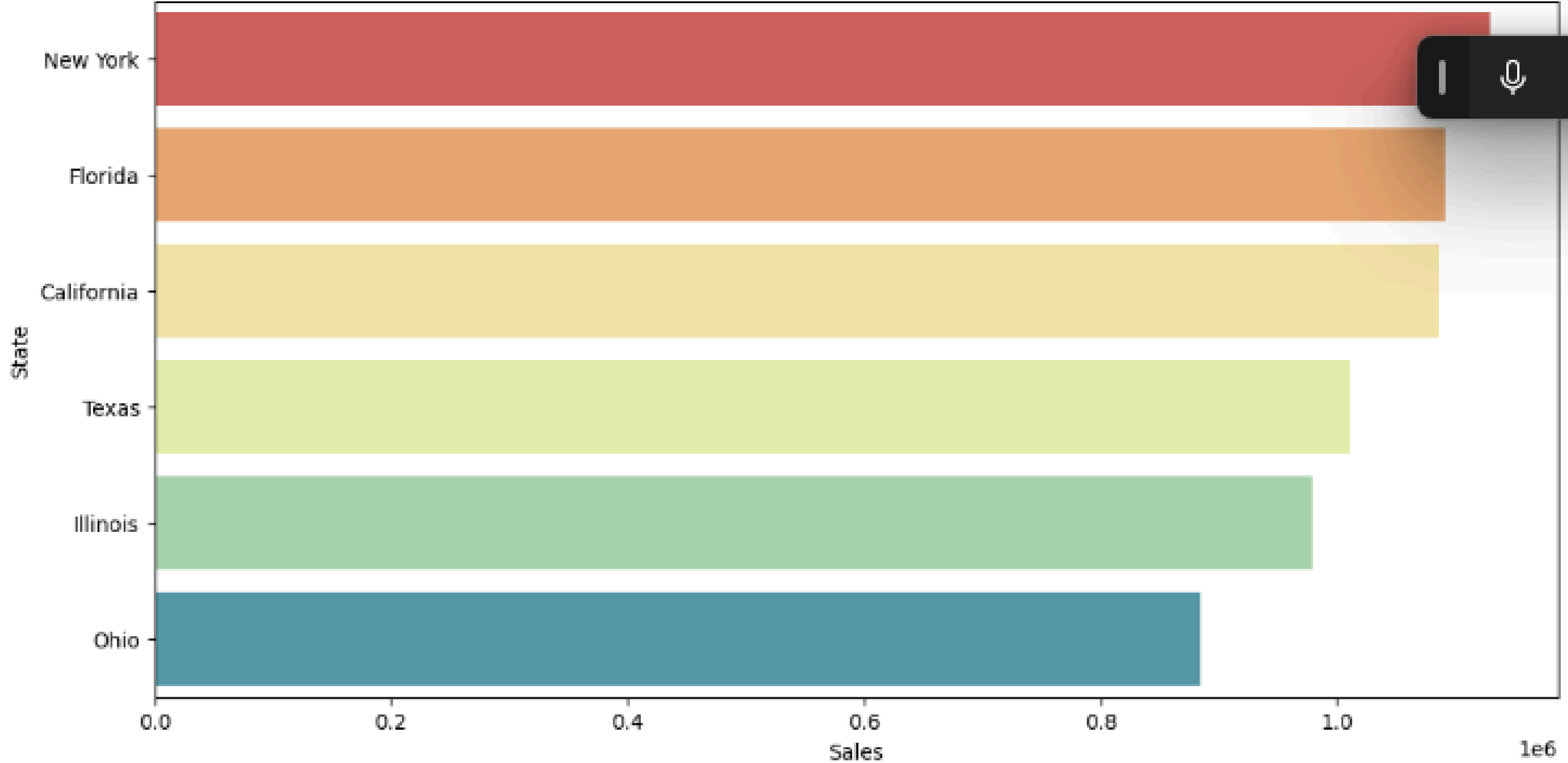
	Amount	Profit	Quantity
count	1194.000000	1194.000000	1194.000000
mean	5178.089615	1348.992462	10.674204
std	2804.921955	1117.992573	5.777102
min	508.000000	50.000000	1.000000
25%	2799.000000	410.000000	6.000000
50%	5152.000000	1014.000000	11.000000
75%	7626.000000	2035.000000	16.000000
max	9992.000000	4930.000000	20.000000

RESULTS

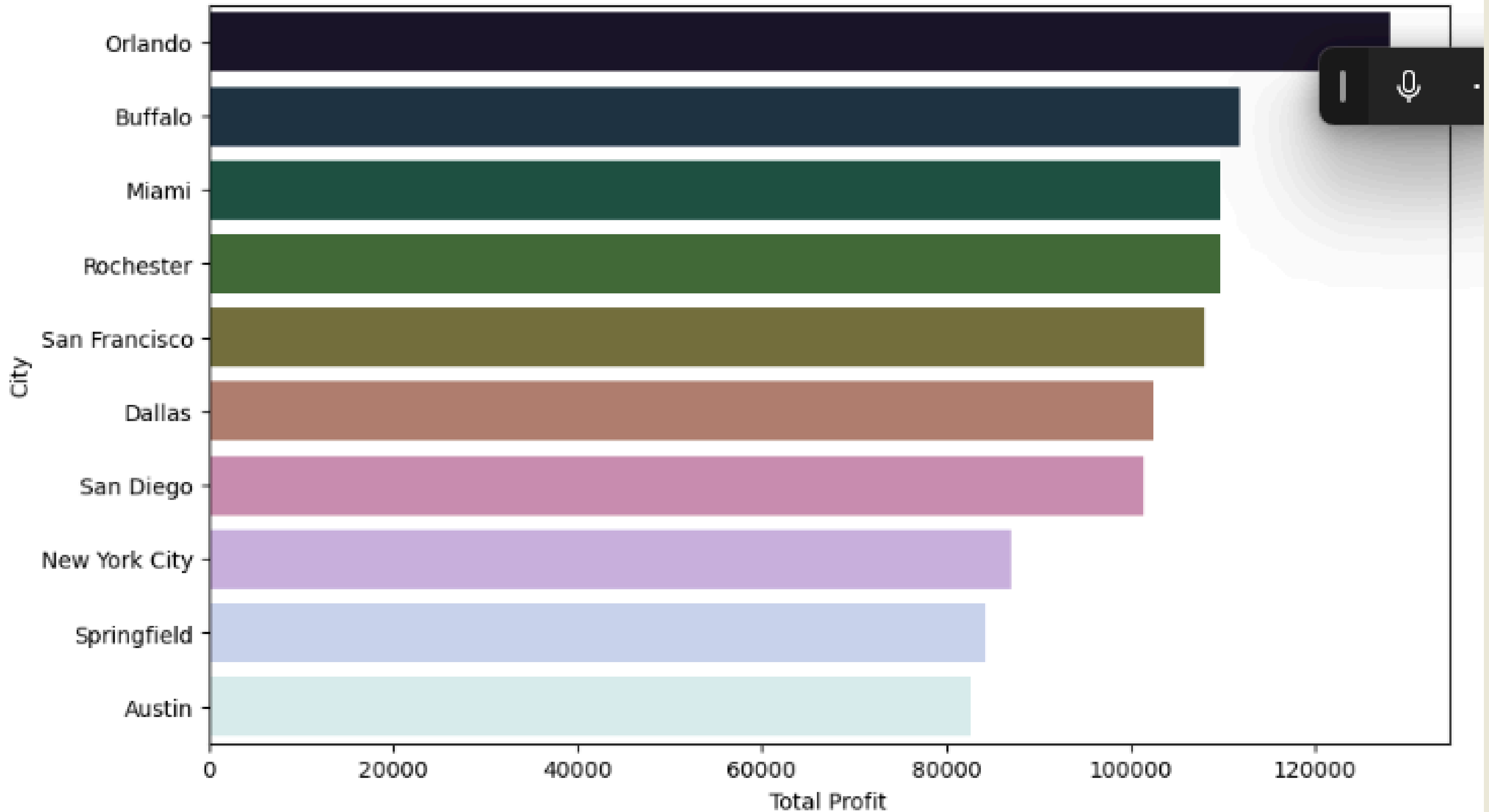
Monthly Sales Over Time

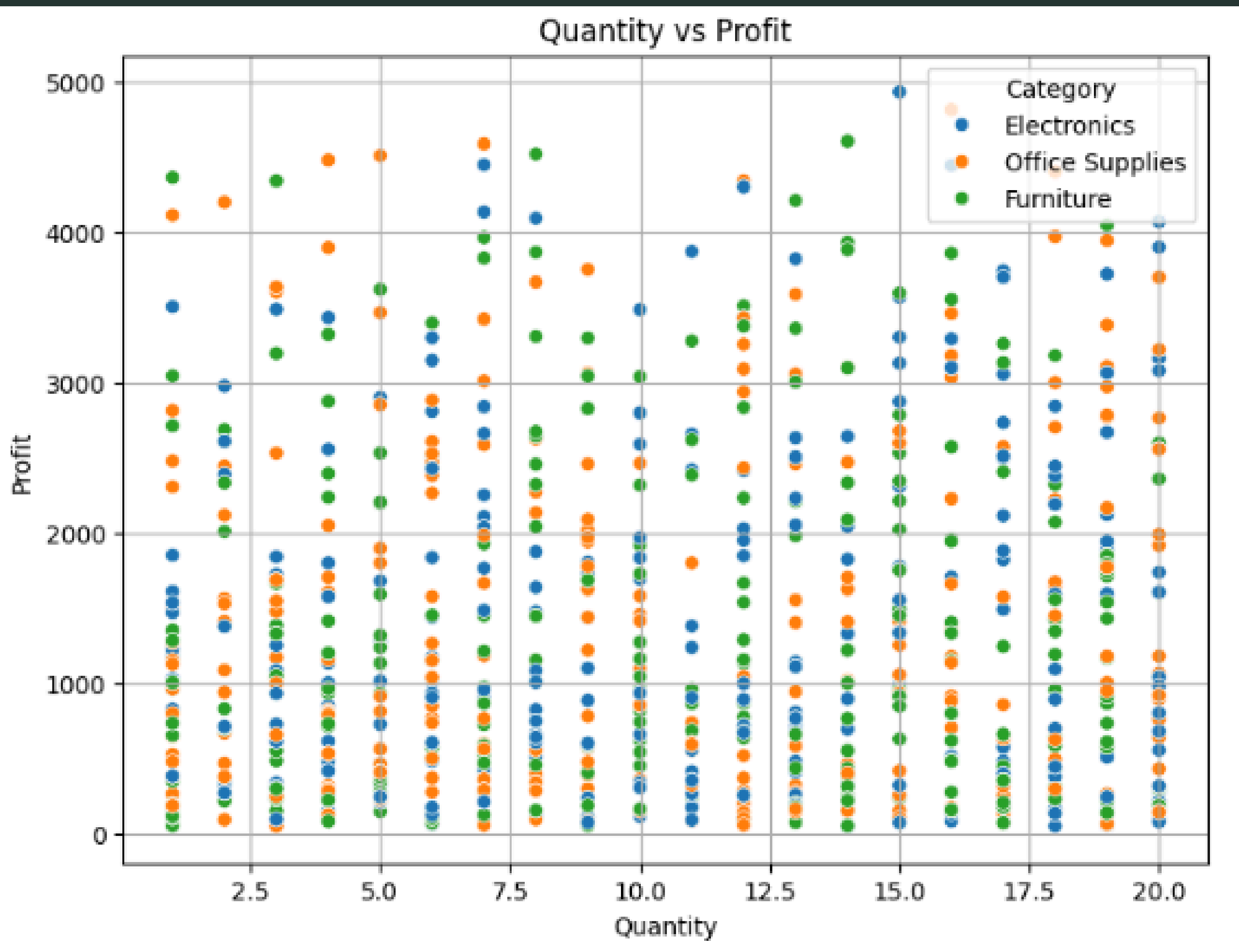


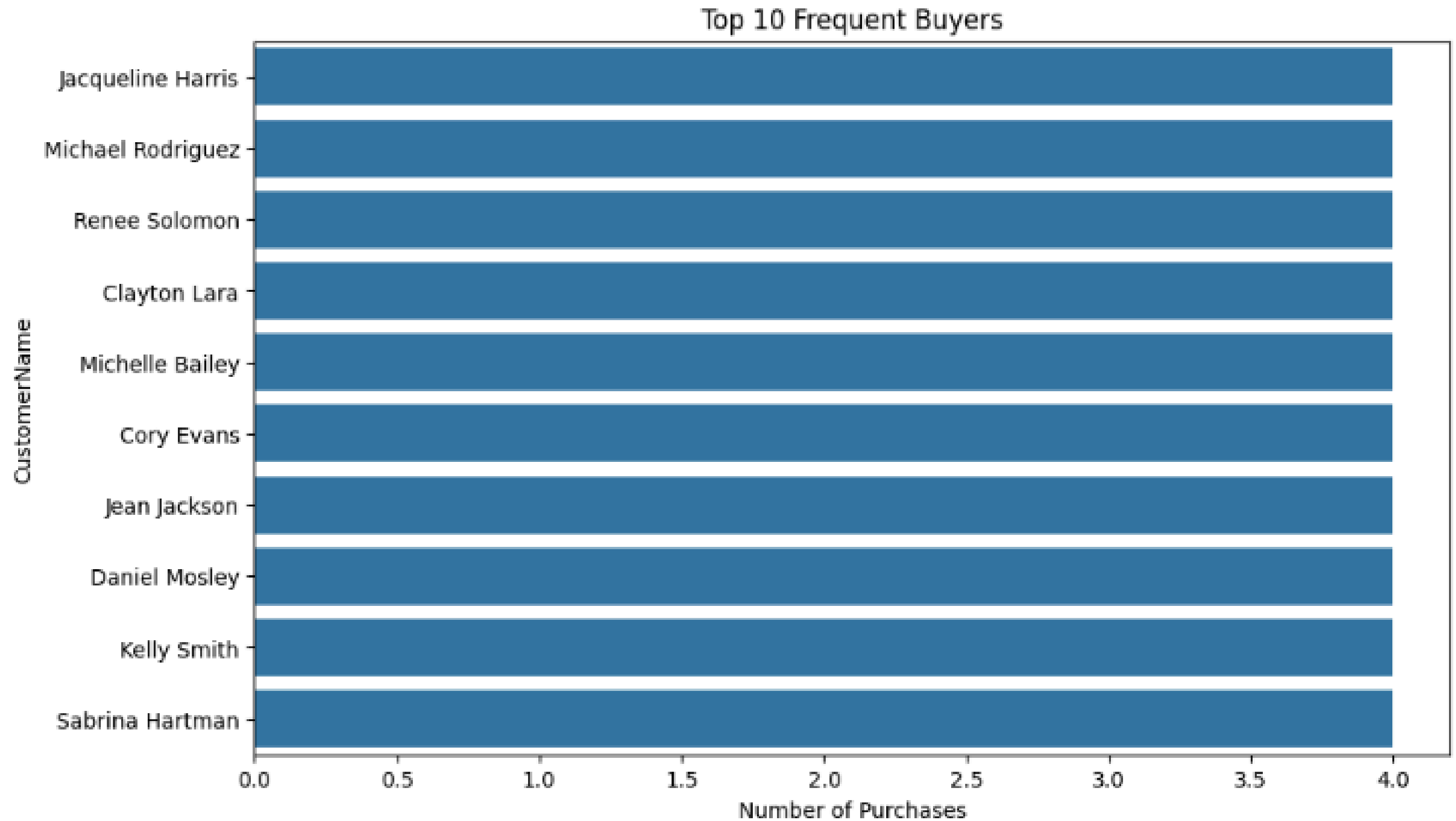
Sales by State



Top 10 Cities by Profit



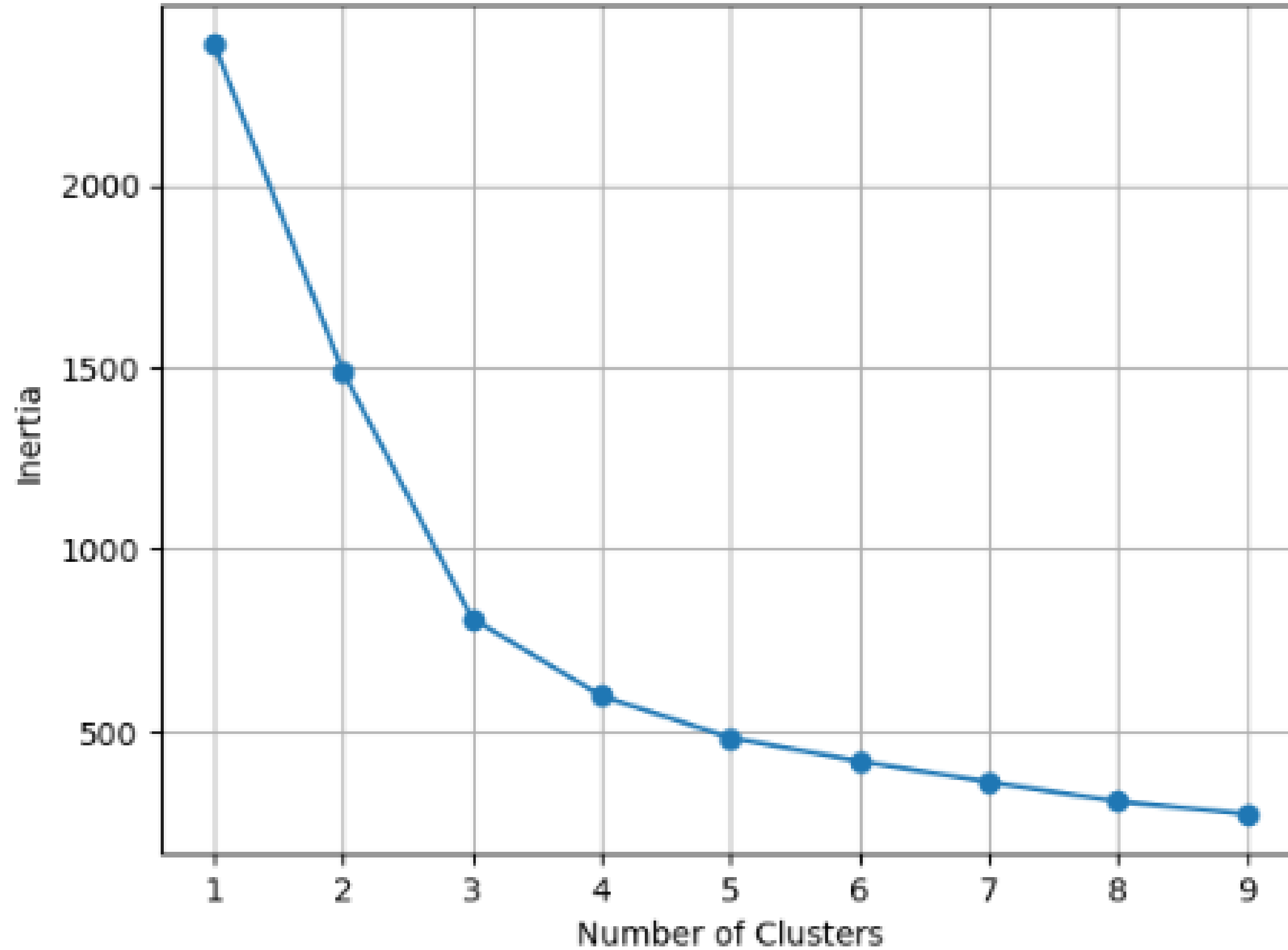




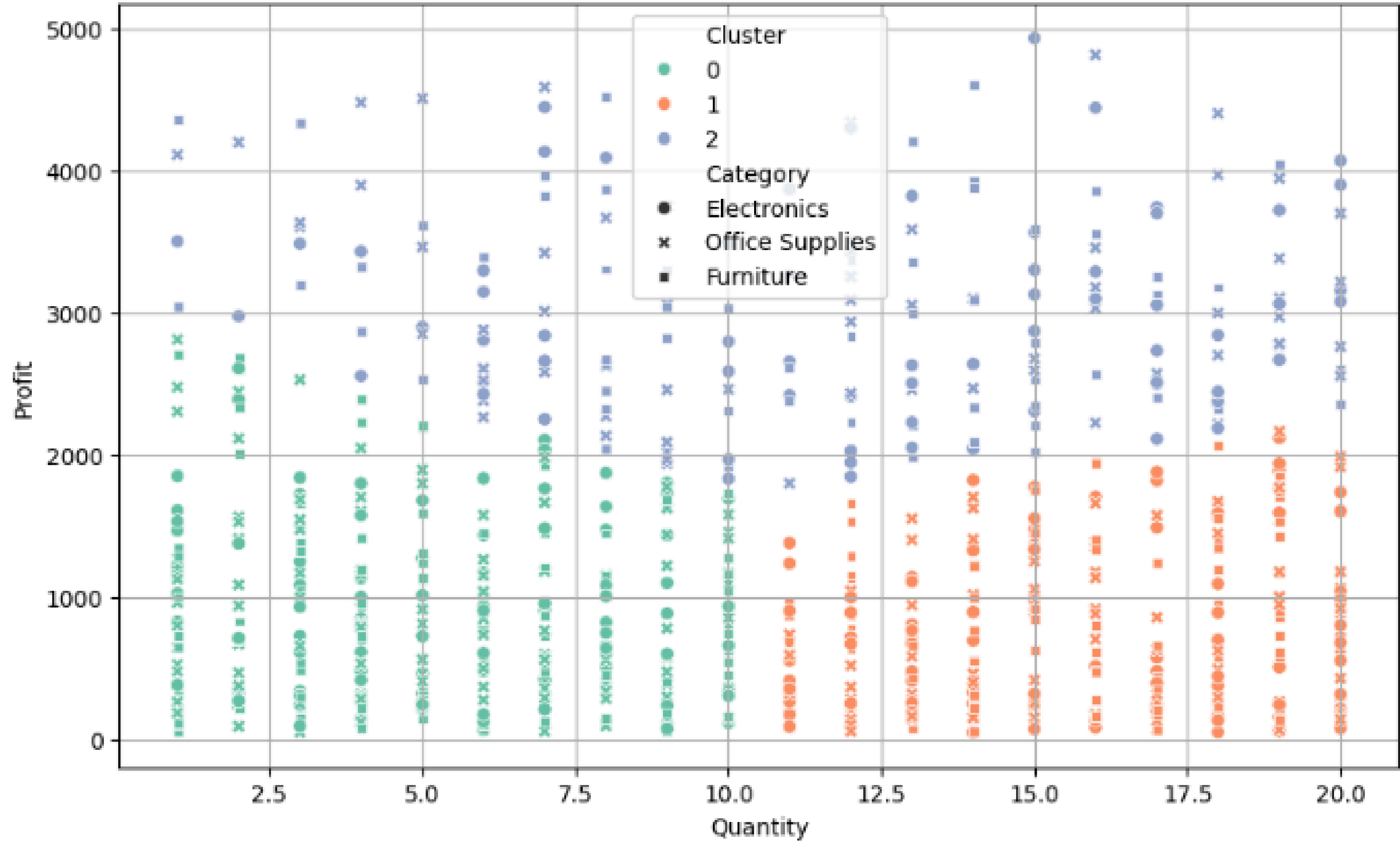
K - MEANS ALGORITHM

- K-Means clustering is a popular unsupervised machine learning algorithm.
- It automatically finds patterns by grouping similar data points together based on their features
- The algorithm divides the data into K distinct clusters, where K is a user-defined number. It works in the following steps:
 - Randomly initialize K centroids (one for each cluster).
 - Assign each data point to the nearest centroid based on distance (usually Euclidean distance).
 - Recalculate the centroids as the mean of all data points in a cluster.
 - Repeat steps 2 and 3 until the centroids no longer change significantly.
- The number of clusters, K, needs to be set manually. One commonly used method to find the optimal K is the Elbow Method, where we plot the Within-Cluster Sum of Squares (WCSS) against different values of K and choose the point where the rate of decrease sharply slows

Elbow Method for Optimal k

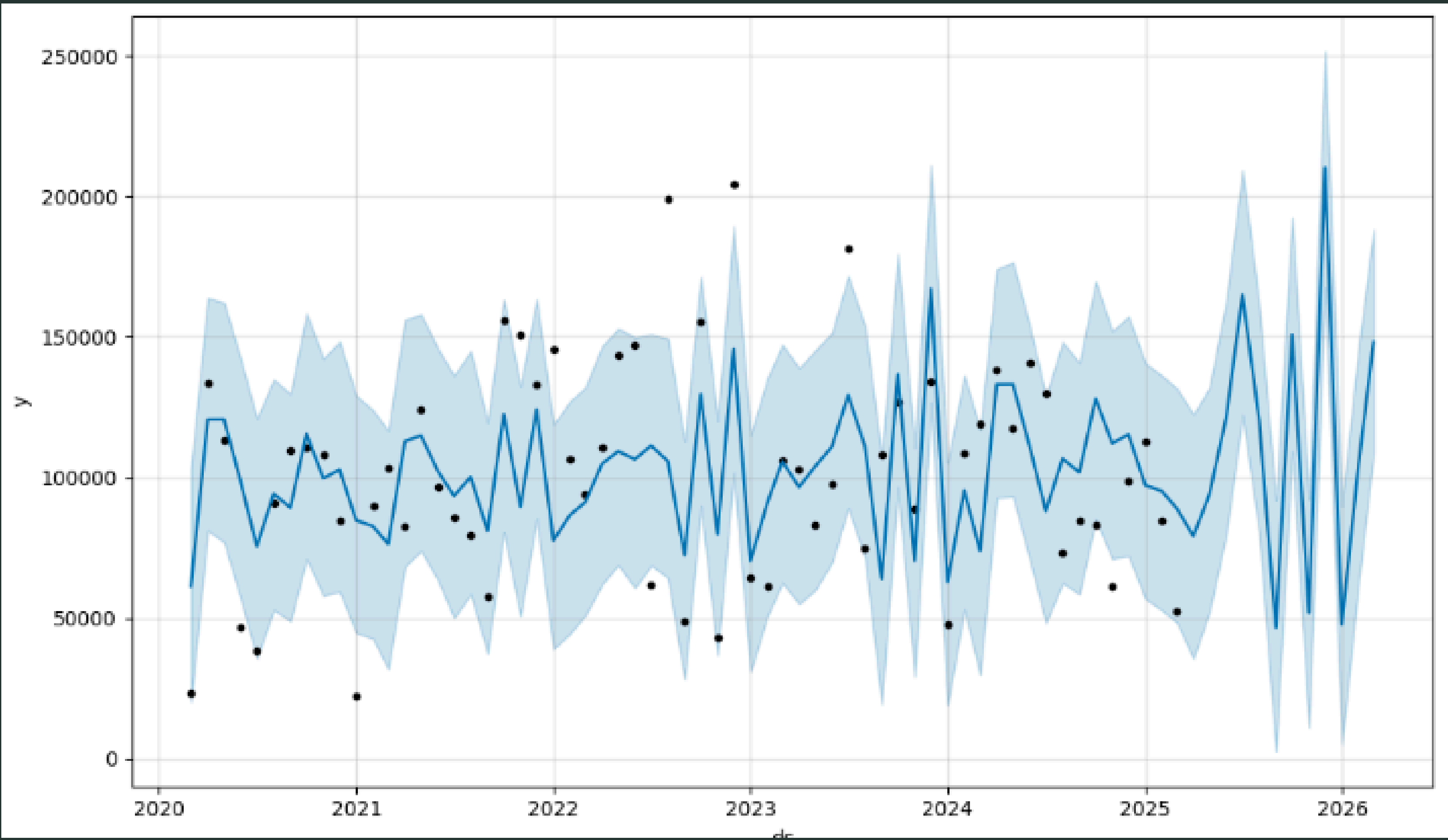


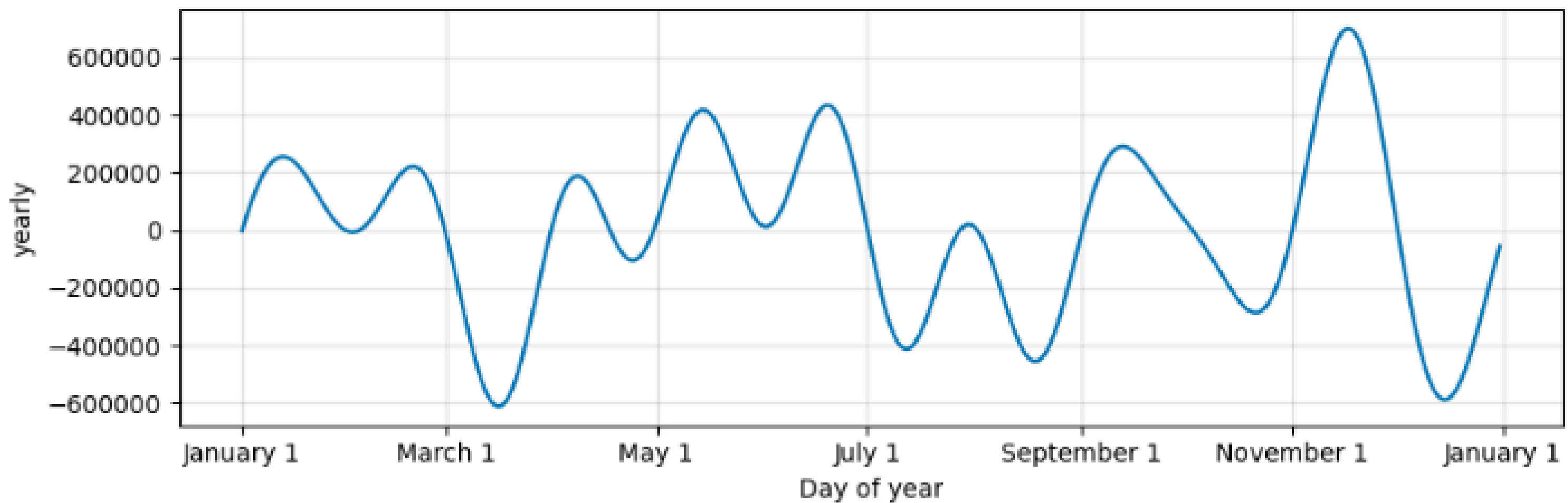
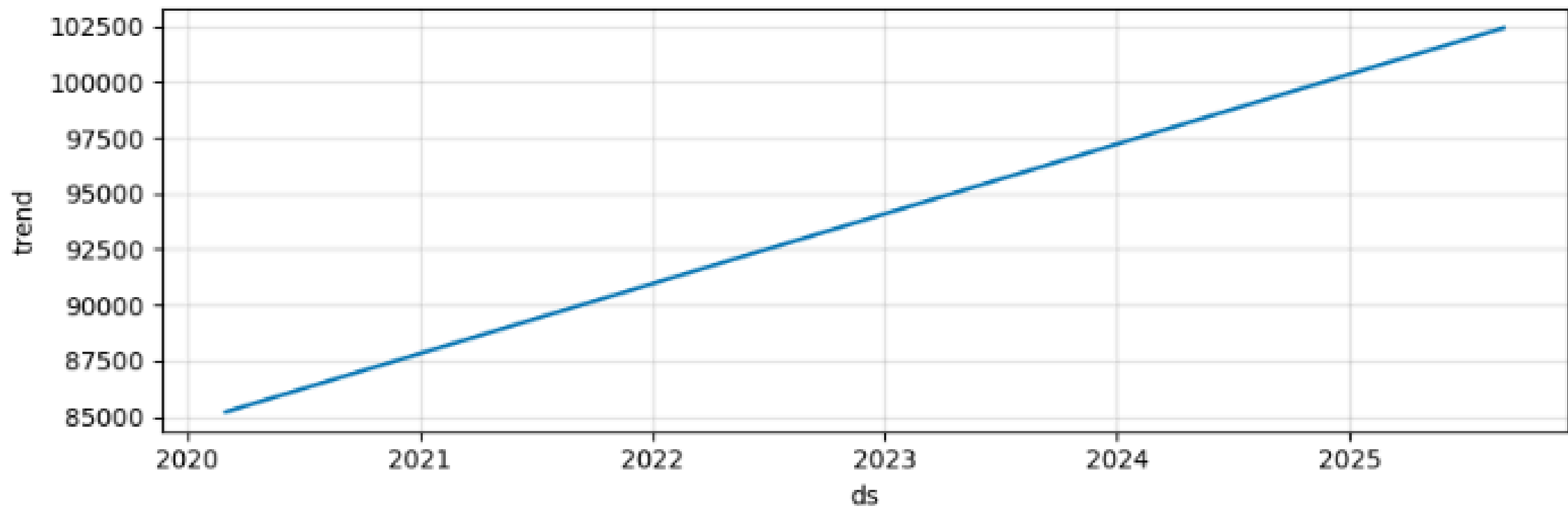
Clusters of Orders (Quantity vs Profit)

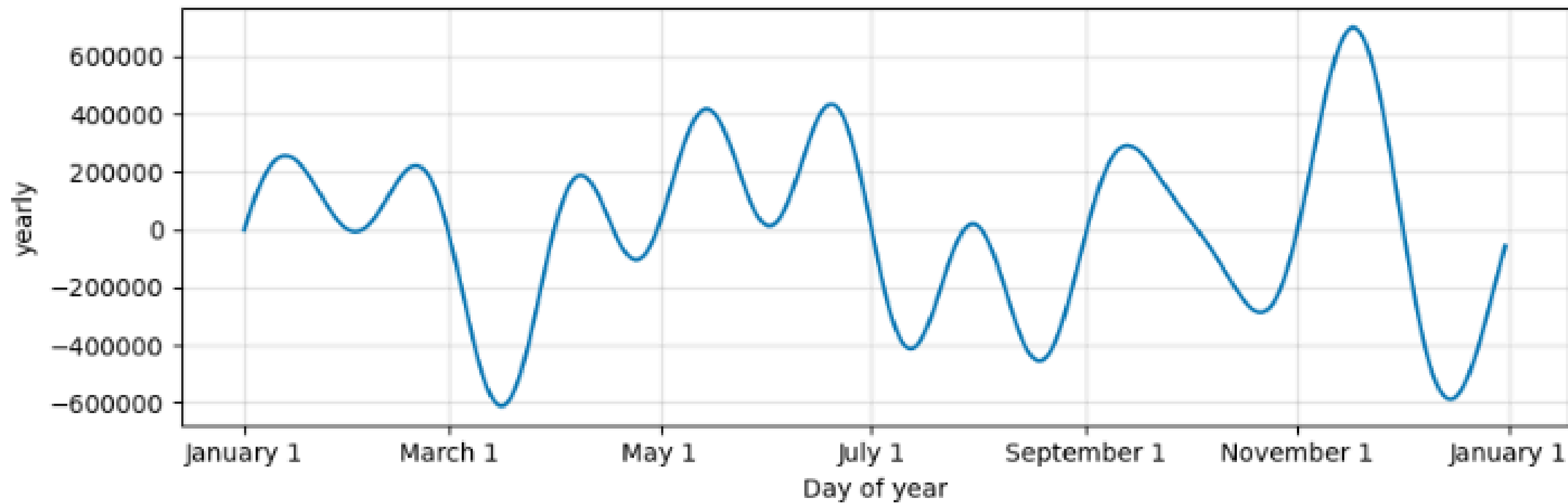
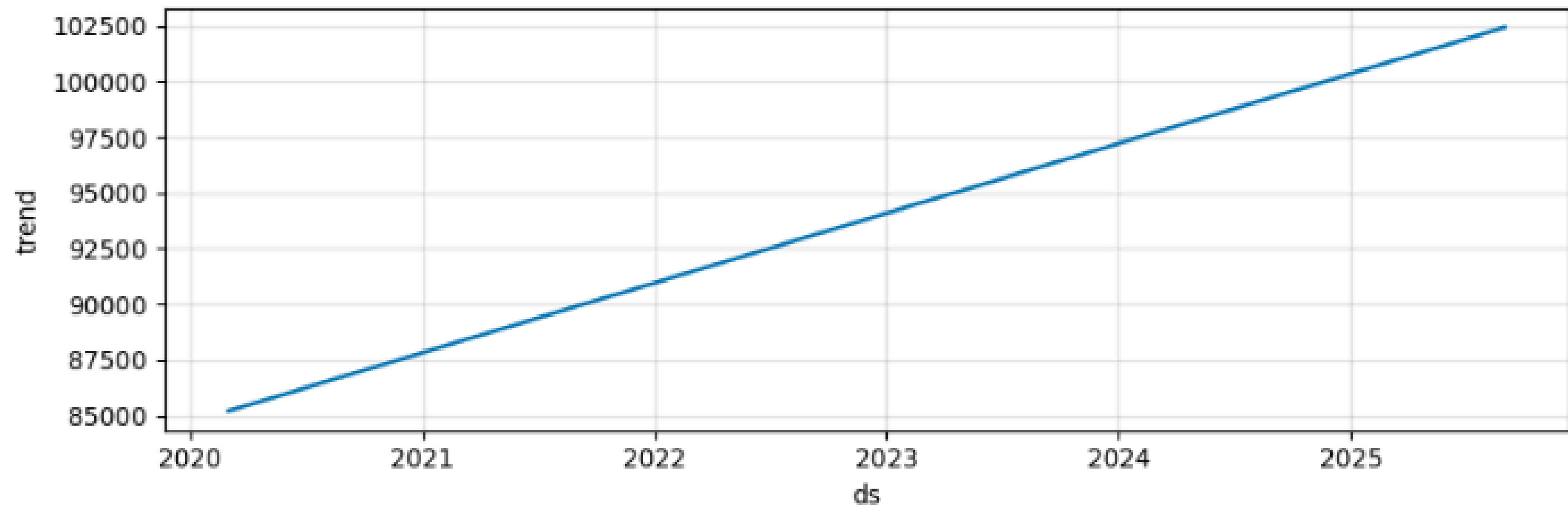


FACEBOOK PROPHET

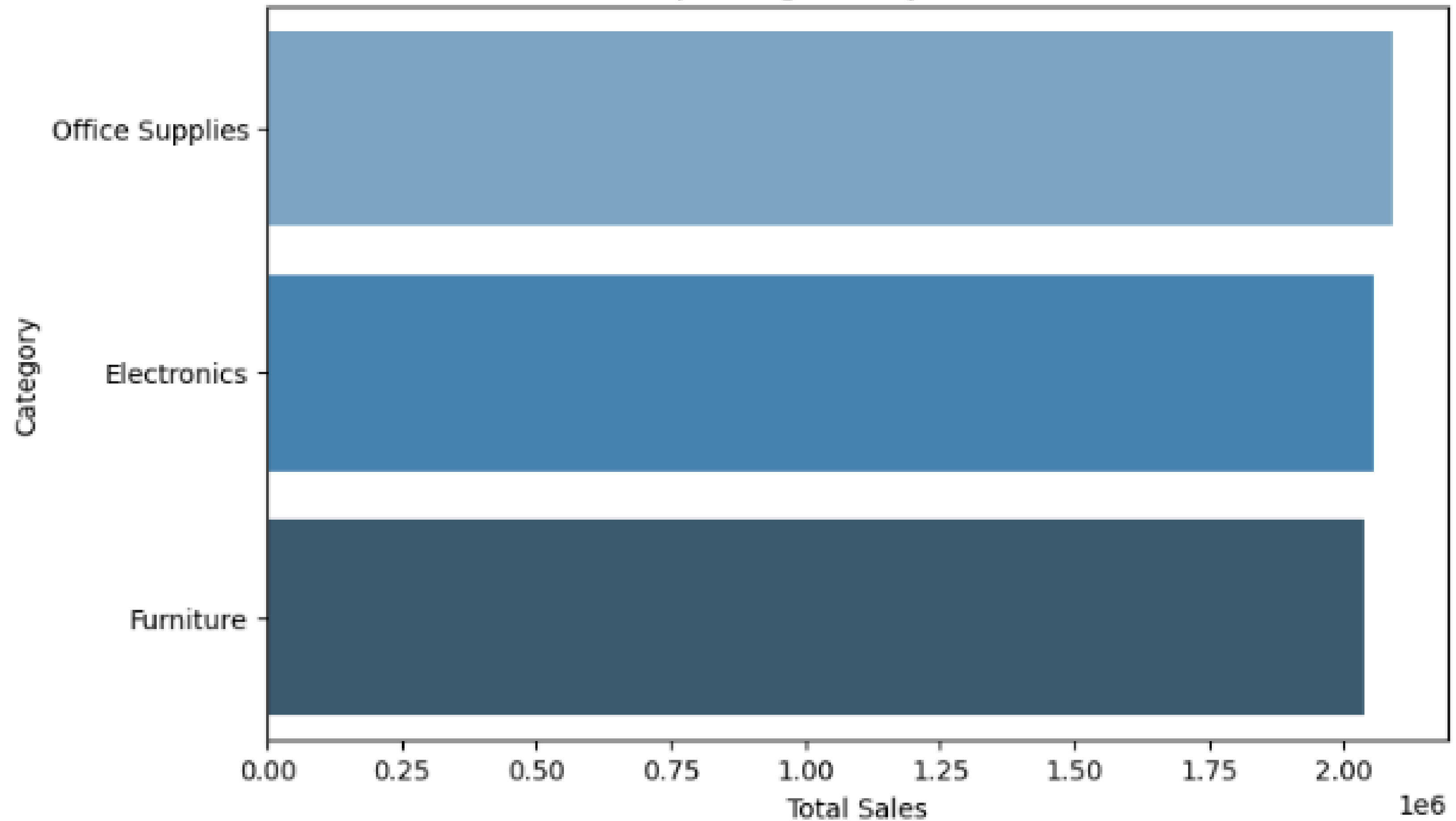
- Facebook Prophet is a time series forecasting algorithm.
- It is specially designed for time series data that shows clear seasonal effects — like sales data, stock prices, website traffic, etc.
- The Prophet model is additive in nature. It breaks down the forecast into components like:
 - Trend – long-term direction of data
 - Seasonality – repeating cycles such as weekends or holidays
 - Holidays/Events – custom or country-specific dates that affect the data
- The user provides the data in a simple format with two columns:
 - ds → the date column
 - y → the numeric value to forecast (e.g., sales, profit, etc.)
- It handles trends, seasonality, holidays, and missing values automatically, making it easy to use even for non-experts.



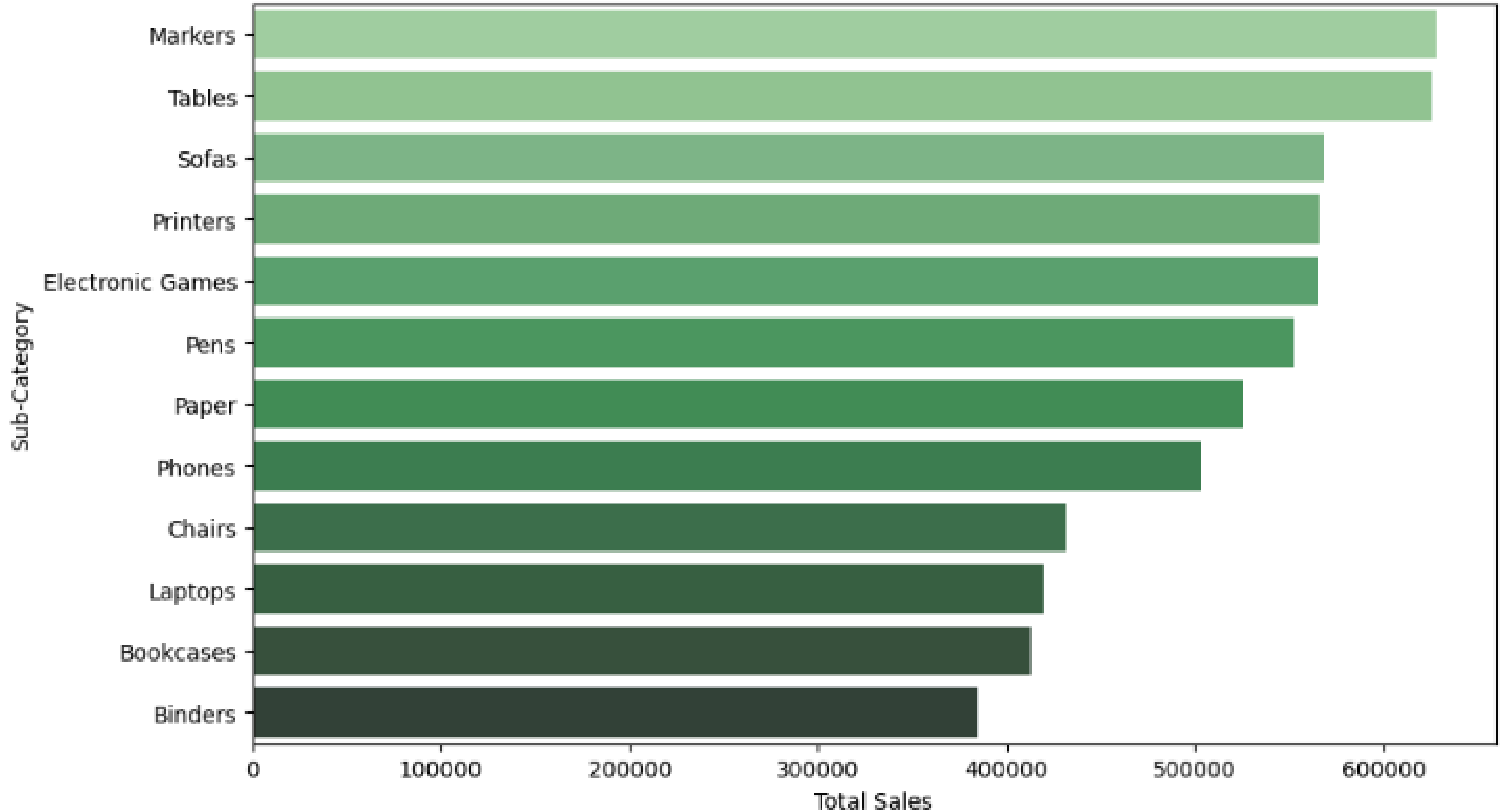




Top Categories by Revenue



Top Sub-Categories by Revenue



Sales and Profit by State

State

New York

Florida

California

Texas

Illinois

Ohio

1130048

1091174

1086436

1011475

978738

884768

Amount

308506

308706

278814

257780

240372

216519

Profit

1e6

1.0

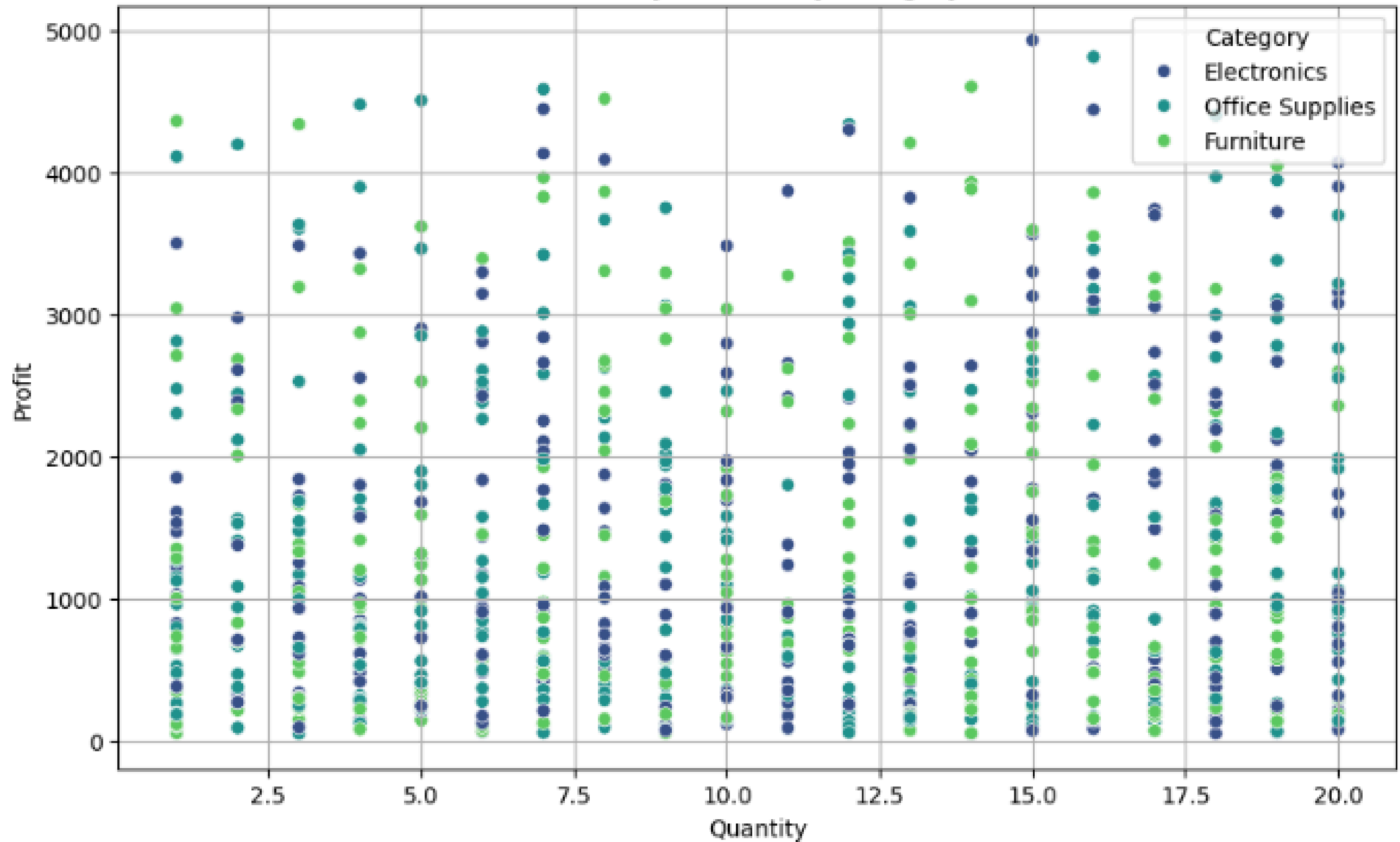
0.8

0.6

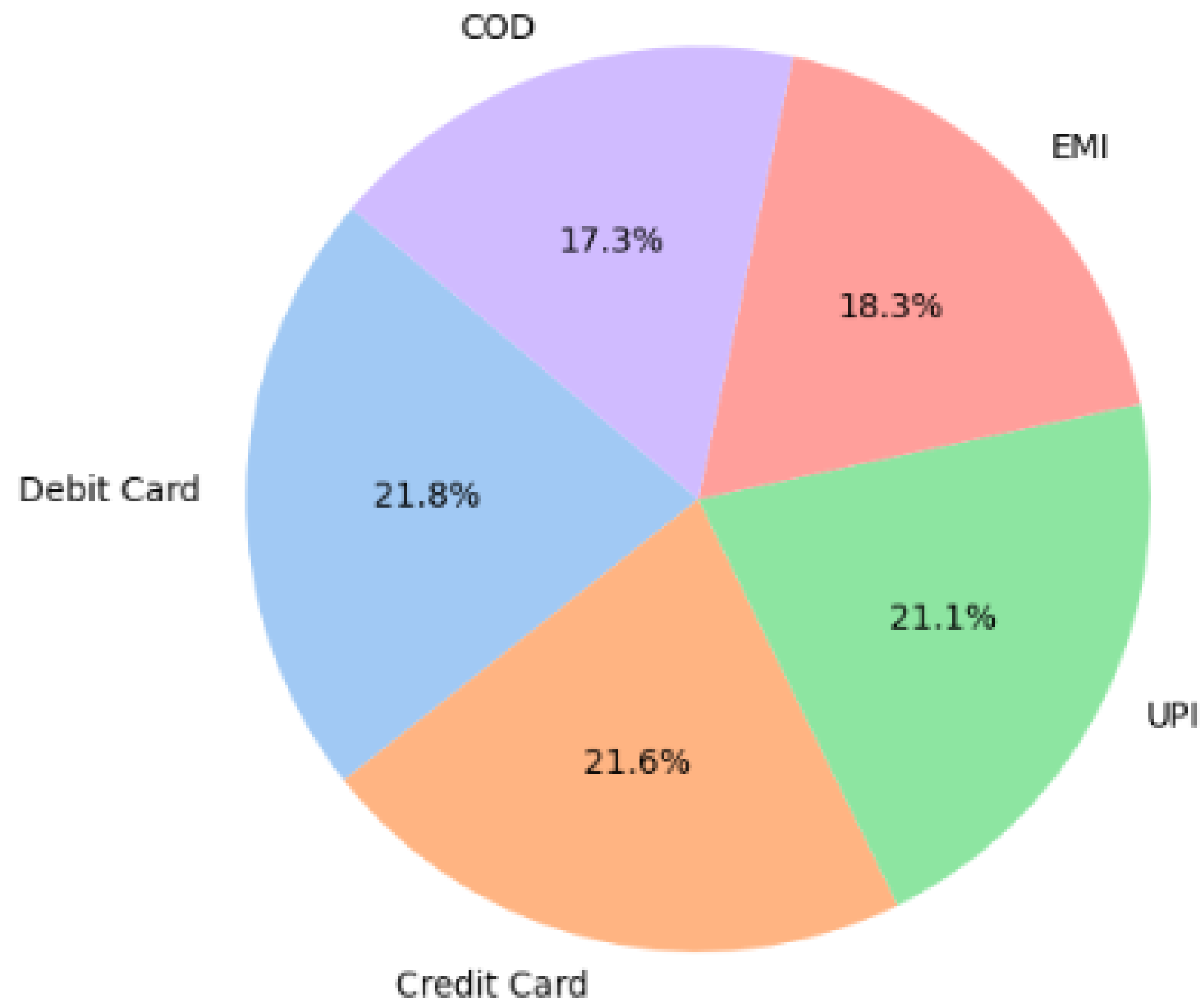
0.4



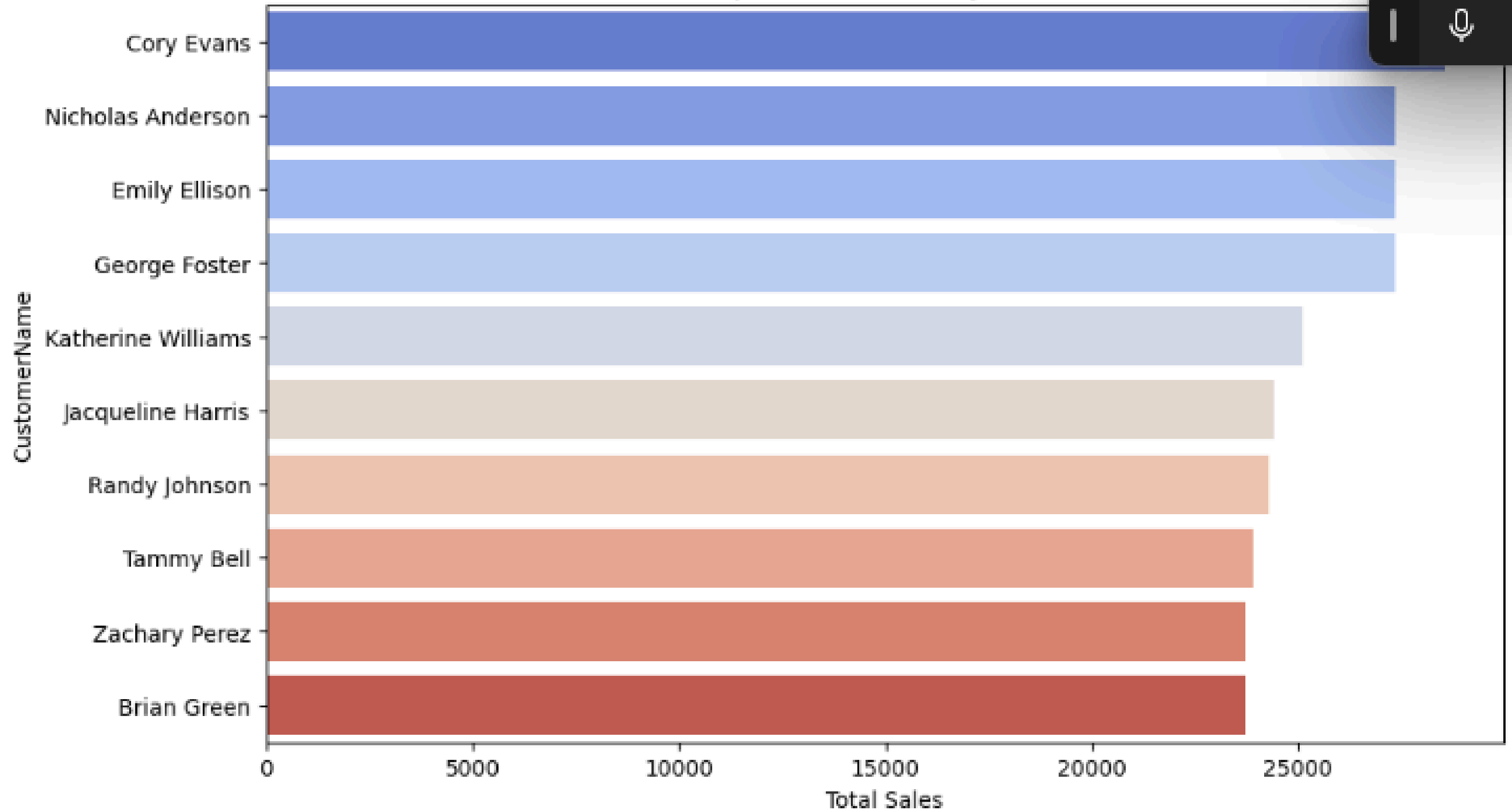
Quantity vs Profit by Category



Payment Method Distribution



Top 10 Customers by Sales



BUSINESS INSIGHTS

- Plan Ahead with Forecasting
Use Prophet to anticipate sales trends and align inventory and staffing.
- Target High-Value Customer
Personalize offers and rewards for top customer segments.
- Expand Geographically
Focus on low-performing states to boost reach and revenue.
- Promote Top Categories
Invest in bundling and cross-selling top products like Phones & Chairs.
- Align Teams with Data Insights
Enable smarter decisions across marketing, ops, and finance

CONCLUSION

The analysis provides a comprehensive view of business performance, highlighting opportunities to optimize inventory, personalize customer engagement, target untapped regions, and improve operational efficiency. Leveraging these insights enables smarter strategies and sustainable growth for the organization.

THANK YOU